

ForgeMind

官方产品与技术文档

DIGITAL FACTORY OS • BUILD 0.1.0 • UPDATED 2026.09.02

这份文档汇总 ForgeMind 及 ForgePass、ForgeCloud、ForgeHub、ForgeLab、ForgeMove 的使用方式、产品能力、系统边界、数据模型和验证规则。内容以当前代码、数据库迁移和已执行验证为准。

阅读地图

从核心用户操作开始，再进入各模块的独立指南和生态技术边界。若只想快速上手，先读对应模块用户文档；若需要扩展、排查或接入服务，阅读核心技术说明和生态模块技术文档。

文档组成

01 核心用户手册 ForgeMind 的启动、登录、建造、生产、仓储、仿真、诊断、存档与排障。

02 核心技术说明 前端状态、确定性仿真、三维渲染、后端、Agent、AI 和数据不变量。

03 模块用户文档 ForgeMind、ForgePass、ForgeCloud、ForgeHub、ForgeLab、ForgeMove 的逐模块使用路径。

04 生态模块技术文档 身份流、职责边界、接口、权限、数据流和验证入口。

ForgeMind 用户使用手册

> 版本：2026-08-26 • 适用 BUILD 0.1.0（存档格式 v6） >

本手册描述的是当前代码的实际行为，所有按钮和界面文字均与源码一致。设计与原理请读《ForgeMind 项目方案.md》与技术文档；本手册只教「怎么用」。

目录

1. 准备与启动 2. 注册登录 3. 认识主界面 4. 十分钟上手：第一条产线 5. 建造详解 6. 楼层系统 7. 生产配置：物品 / 路线 / 机械制造 8. 仓储与物流 9. 运行仿真与监控
- 9.4 视觉检测 Demo：视频实时 YOLO 识别
10. AI 诊断页 11. 存档与数据安全 12. 可选 AI 与语音 13. 快捷键与鼠标速查 14. 常见问题排查 15. 当前已知限制

1. 准备与启动

1.1 环境要求

项目	要求
操作系统	Windows 10/11
Node.js	≥ 18（前端构建与开发服务器）
JDK	17（后端 Spring Boot）
Docker Desktop	运行 MySQL 8.4 项目实例（可用 -SkipMySQL 跳过）
浏览器	Edge / Chrome 现代版本

1.2 一键启动（推荐）

双击项目根目录的 启动ForgeMind.cmd。它会依次：

1. 启动 MySQL 8.4 容器并执行 Flyway 数据库迁移； 2. 编译并启动 Spring Boot 后端（http://127.0.0.1:8080）； 3. 启动 AI 服务 FastAPI（http://127.0.0.1:8000）； 4. 启动前端 Vite 开发服务器（http://127.0.0.1:5173）并自动打开浏览器。

每个服务都在独立窗口中运行；已监听的端口会被复用而不会重复启动。全部就绪后你会看到各窗口打印端口信息。

常用变体（在命令行执行 scripts\start-forgemind.ps1 时附加参数）：

参数	作用
-SkipSpring	不启动后端与数据库（纯前端演示，无法注册登录保存）
-SkipMySQL	跳过数据库容器（后端仍启动但需要已有数据库）
-NoBrowser	不自动打开浏览器
-ForceRebuild	强制重新安装依赖/构建

1.3 停止

关闭方式二选一：

- 直接逐个关闭各服务窗口；
- 或运行 scripts\stop-forgemind.ps1，它会结束 Vite、Spring、FastAPI 进程并释放 5173 / 8080 / 8000 端口。数据库容器不会被删除，下次启动复用。

> 提示：直接关机前建议先跑一次停止脚本，避免残留 node/java 进程占用端口。

1.4 首次进入的样子

浏览器打开 <http://127.0.0.1:5173> 后先看到产品介绍页（顶部导航：产品工业 / 技术核心 / 运行流程 / 文档发布）。点击任意 [进入工作台 ↗](#) / [进入数字工厂 ↗](#) 按钮，进入认证页。

2. 注册登录

认证页标题为 舰桥通行认证。

1. 第一次使用请点 02 首次登记 标签。2. 填写两个字段：

- 驾驶员 ID（即用户名）：至少 2 个字符；
- 访问密钥（即密码）：至少 6 位。

3. 点击 登记并进入基地。成功后播放欢迎音效并过渡到工厂界面；之后再用 01 已有身份 → 授权并开启舱门 登录同一账号即可。

常见错误提示：

提示	含义
驾驶员 ID 至少需要 2 个字符	用户名太短
访问密钥至少需要 6 位	密码太短
认证链路中断，请检查基地连接	后端 8080 没起来，回到第 1 节检查启动

> 登录成功后还会弹出项目弹窗（见第 11 节），必须新建或打开一个工厂才会真正进入可编辑的工作台。

3. 认识主界面

顶栏: DIGITAL FACTORY / 工厂名 N层 · 后端存档 ? 用户 用户			
左		右侧面板	
侧 3D 视口		FACTORY	
导 (视口头部: 视图编号/CAMERA 读数)		STATUS	
航 (单选设备时右侧滑出设备详情短窗)		SimPanel	
(底部: 快捷键条 + 仿真运行状态灯)		产出/消耗	
底部视图坞: 01总览 02建造 03生产 04诊断 (+辅助坞 05~08)			
KPI 条: 生产效率 · 设备利用率 · 实时产出 · 物流负载 · 管家球			

3.1 四个主视图（左侧导航 / 底部视图坞）

编号	名称	用途	相机
01	总览	日常监控与操作的主视图，唯一显示 KPI 条和辅助坞	ISOMETRIC 等轴测
02	建造	放置设备、拖绘传送带	TOP-DOWN 俯视
03	生产	沿生产链查看物料流向、机器状态与实时产出（独立整幅控制台）	ISOMETRIC
04	诊断	AI 工厂诊断：Agent 分析、自动巡检、生成式规划	TOP-DOWN

3.2 辅助工作区坞（仅总览视图出现）

编号	名称	内容
05	机械制造	机器定义 CRUD 与机器模型库
06	生产路线	配方目录 / 详情 / 工艺链三栏总览与编辑
07	物品详情	业务物品 CRUD 与 ForgeCore 参数化模型库
08	货物仓储	库存控制 + 导航控制（AGV/无人机编程）

所有工作区都是覆盖层，右上角 × 或按 Esc 关闭。按 Esc 还会在非总览视图时退回总览。

3.3 其他常驻元素

- KPI 条四卡：生产效率（实际/理论产出速率）、设备利用率（运行台数/总台数）、实时产出（units/min，含本次会话累计）、物流负载（占用槽位比例）。
- 智能管家球（AssistantOrb）：状态 待命/聆听中/分析中/播报中/链路异常；旁边是语音输入按钮（详见第 12 节）。
- 右侧数据面板：设施 / 物品 / 配方 / 物流四个小统计 + SimPanel（见第 9 节）。
- 顶栏 ☒ 设置：包含 渲染目标帧率（60 FPS 画质优先 / 120 FPS 流畅优先），帧率选项保存在本机；其余界面设置仅当前会话生效。
- 视口工具条：坐标读数、GRID/1M、相机操作提示（可在设置里关闭）。

4. 十分钟上手：第一条产线

空白工厂是真正的零设备开局——物品、配方、机器都要自己建。跟着做一遍就理解全流程了。本教程做一条最简单的线：入货仓库供铝锭 → 数控机床加工成支架 → 出货仓库收货。

第 1 步：建厂 登录后在项目弹窗输入名称，点 + 新建空白工厂（此时它只是未保存草稿）。

第 2 步：创建两种物品 打开底部 07 物品详情 → 在模型库里搜索 ingot（或任意原料模型）→ 点 使用此模型创建物品：

- 专属 ID alu_ingot，名称 铝锭，分类选基础形状类，保存；
- 同样再创建 bracket / 支架（可换一个零件模型）。

第 3 步：定义工艺 打开 06 生产路线 → + 新建工艺：

- 路线 ID r_bracket，名称 支架加工，加工时长 6 秒，勾选启用；
- INPUT 侧选择物品 铝锭、数量 1，点 + 添加；
- OUTPUT 侧选择 支架、数量 1，添加；
- 点 加入生产路线。

第 4 步：定义机器 打开 05 机械制造 → + 新建机器：

- 机器 ID cnc_1，名称 数控车床，模型来源选内置 数控加工中心（占地 3×2）；
- 蓝色入货口数量 1、黄色出货口数量 1（每侧接口数不能超过占地深度 2）；
- 在右侧 录入已有工艺路线 勾选 支架加工；
- 保存。现在建造菜单的「B 基础加工」分类里就会出现这台机器。

第 5 步：摆放与连线 切到 02 建造 视图：1. 「A 货物仓储」放一台 入货仓库（3×3）；「传送物流」再放出货仓库（3×3）和你的 数控车床，三者留出间隔。2. 选滚筒输送线，从入货仓库前方开始按住左键沿网格拖绘到机床的蓝色入口旁松手（靠近兼容端口会自动吸附对齐）。3. 再拖一条从机床黄色出口到出货仓库的带子。方向不对的段落：选中后用详情窗 TRANSFORM 旋转 90° 或重画。4. 点一下入货仓库，在设备详情窗 INBOUND SUPPLY 无限供应物品 里选择 铝锭。

第 6 步：绑定配方并开机 点击机床 → 设备详情窗 生产配方 RECIPE 下拉选择 支架加工 → 回到总览 → 右侧 SimPanel 点启动。

第 7 步：看结果

- KPI 条出现产出速率；右侧 OUTPUT 列表开始累积 支架；
- 打开 08 货物仓储 → 物料台账：入货仓库取出量记为消耗，出货仓库入库量记为产出；
- 满意后点顶栏 手动保存存档。

5. 建造详解

5.1 进入与通用操作

底部视图坞点 02 建造，左侧弹出 BuildMenu 目录。顶部提示原文：

> 网格 = 1 m • 蓝色端口为入口 • 琥珀端口为出口 • 平面输送带左键拖绘自动转弯 • 跨层与平面输送带靠近兼容端口时自动吸附

操作	效果
选一个设备卡	出现半透明预览（ghost），跟随鼠标吸附网格
移动鼠标	ghost 绿框=可放置，红框=越界/重叠/非法
R	旋转 90°
左键	确认放置
右键	取消当前建造工具
Esc	退出建造模式

5.2 可放置对象全表

A 货物仓储

对象	占地	说明
货物存取站	4×4	三面各有一个 2×2 货架泊位，站内自带短传送带，负责货架↔传送带双向装卸
货物仓储架	2×2	普通有限容量货架，AGV/无人机可直接取放
入货仓库	3×3	无限供货起点，取用量计入消耗台账
出货仓库	3×3	无限接收终点，入库计入产出台账且不可取回

B 基础加工：你在机械制造里定义的机器都会出现在这里；目录为空时提示先去机械制造新建。

C 精密装配

对象	占地	说明
机器人装配单元	3×3	多输入合并为一输出的装配站，7 轴 Panda 机械臂
双臂视觉质检单元	2×2	双臂质检，可进独立 360° 检测工作台

D 传送物流

对象	说明
滚筒输送线	平面传送带，左键拖绘，自动补格转点
向上跨层输送线 / 向下跨层输送线	跨层坡道，需与其他带或端口同向吸附对接
三向分流器 / 汇流节点	分发与合并物料流
AGV 叉车搬运车	地面八方向寻路运输车
货运无人机	26 邻域三维 A* 任意跨层飞行运输

5.3 传送带拖绘规则（重要）

1. 选中输送线后，按住左键拖动即开始绘制，经过的格子自动补齐； 2. 拖绘路径同时改变了行列两个轴时，按上一轴优先补出直角转弯； 3. 往回拖过上一格 = 擦除那一格（可以边画边改）； 4. 拖回早前的格子会裁掉中间形成闭环的多余段，不会自叠； 5. 松开左键时整条路径统一校验后落位； 6. 带端靠近机器蓝/黄端口或其他带的兼容接口时自动吸附对齐； 7. 跨层坡道只能在与平面带兼容的方向上对接，上下楼层各有一段落地端。

5.4 放置后的调整

点击任意设备打开右侧设备详情短窗，可：查看占地/朝向/吞吐、绑配方、改仓储设置、TRANSFORM 旋转 90°、REMOVE UNIT 拆除设备。多选时出现 MULTI SELECT 已选择 N 个对象 徽章，DELETE 批量删除，点击空白取消。

6. 楼层系统

右侧（或左侧竖轨下方）的 VERTICAL GRID 楼层面板：

- 切换楼层：直接点击 L1/L2/…
行。被点的楼层立即变为活动层并成为唯一可编辑层，同时清除当前建造工具与选中。
- 加层：点底部 + 添加楼层，上限 12 层（达上限按钮禁用）。
- 命名楼层：每行的名称就是输入框，直接改名即时生效；默认名依次为 生产层 / 工艺层 / 装配层，之后是 扩展层 N。
- 上下文显隐开关（眼睛图标）：把其他层设为「只读上下文」——只显示简化建筑轮廓和承托平台，无网格、不可选、不可改，用于跨层对位参考。当前层的开关在切走后才生效。

活动层有唯一的 1m 玻璃网格地面；层高 5.25m，向上/向下跨层靠第 5.3 节的坡道输送线连接。AGV 只在各自楼层的平面移动；无人机可以在任意楼层之间直线三维飞行（见 8.6）。

7. 生产配置：物品 / 路线 / 机械制造

三者职责严格分离：物品详情维护「有什么货」，生产路线维护「怎么变换」，机械制造维护「用什么机器」。后两者只能引用前者已存在的内容。

7.1 物品详情（07）

- 创建物品：模型库（36 个可参数化模型，支持搜索/分类/分页）→ 使用此模型创建物品 → 填写表单。
- 表单字段：专属 ID（唯一）、业务编码、物品名称、分类、基础模型、显示颜色、单位质量 kg、最大堆叠数、说明。质量与堆叠只是业务属性，不影响网格。
- 建模参数（右侧栏）分两组：结构与几何 / 材质与表现；部分参数有依赖条件（会提示 需满足：…）；☒ 恢复默认一键还原。
- 删除保护：物品被生产路线或场景设备引用时删除按钮禁用，tooltip 会列出引用处；未引用才可安全删除。

7.2 生产路线（06）

- 三栏布局：配方目录（参与仿真/已停用）→ 配方详情 → 当前工艺链。
- 新建/编辑走覆盖层表单：ID、编码、名称、加工时长（秒）、说明、启用勾选，左右两侧分别是输入/输出物品列表（下拉+数量，可多条）。
- 常见校验错误对照：

报错	原因
请先在"物品详情"中创建物品	还没有任何物品
工艺 ID、名称、输入和输出都不能为空	必填缺失
同一侧不能重复选择同一种物品	输入或输出列表里有重复物品
工艺路线 ID 已存在	ID 冲突

- 停用的路线不会出现在机器录入候选里；基础加工机器只能录入端口容量相容的路线（见下）。

7.3 机械制造（05）

- 机器仓库：已定义机器卡片（N 入/N 出、工艺 N 条、场景中 N 台实例）；有实例时禁止删除定义。
- 新建机器字段：专属 ID、名称、模型来源（内置 7 种或你导入的 GLB）、模型高度 m、占地宽/深（格）、蓝色入货口数、黄色出货口数、标准吞吐、额定能耗、说明；右侧勾选要录入的工艺路线。
- 规则：每侧接口数量 ≤ 占地深度；已录工艺的输入种类数不得超过入货口数（输出同理），不相容的路线会标超出接口容量并禁选。
- 导入自己的 GLB：机器仓库顶部 ☒ 导入模型 打开导入弹窗（JSON 定义 + GLB 模型，字段见弹窗提示）；登录状态下导入的资源会保存到你个人账号名下，只有你能看到和使用。

8. 仓储与物流

8.1 四种仓储设施一览

设施	库存语义	台账口径
货物存取站	自身不存货；三面泊位吸附真实货架对象	通过它进出货架
货物仓储架（普通货架）	有限容量真实库存，可命名 40 字	AGV/无人机/存取站都按实际数量取放
入货仓库	无限供货，物品由你指定	实际取出多少记多少消耗
出货仓库	无限接收，入库后不可逆	实际送达多少记多少产出

8.2 货物存取站程序

点击存取站 → 详情窗 CARGO ACCESS PROGRAM • 3-SIDE RACK：

1. 三面状态行显示 后侧/左侧/右侧 是否已吸附货架（已吸附 {id} / 未连接）； 2. 运行模式二选一：取货 → 传送带（把货架货送上带）或 传送带 → 存货（把带来货存上架）； 3. 存取间隔（秒，0.25-60）控制节拍； 4. 取货模式下指定当前取货物品； 5. 物品 → 实际货架侧映射：默认 自动分配并同类合并，也可手动指派到某一侧货架。缺架时：取货会阻塞等待；存货则保留传送带来货不丢。

8.3 普通货架

- 详情窗设置 货物上限 和每种物品的 仿真初始数量；
- ACTUAL INVENTORY 实际库存 实时显示 N / 容量；
- 总览视角下货架上方有浮动标签（名称、物品 ×N、总量/容量）；打开任何工作区/建造/多选时标签自动隐藏避免遮挡。

8.4 AGV 编程

入口两处：选中 AGV 的悬浮窗，或 08 货物仓储 → 导航控制 页。任务结构固定为三节点：

1. 01 起点 装货位置：下拉选择（普通货架、入货仓库等，优先显示自定义名称）； 2. 02 货物 搬运内容：选物品 + 每趟数量； 3. 03 终点 卸货位置。

可选 ROUTE DESIGNER 中间站：添加站点并指定动作 经过 / 装货 / 卸货，可排序删除；动态遇障时保留任务顺序绕行重规划。

发货设置：

设置项	说明
供货方式	一直运输（循环搬）/ 库存条件触发（满足阈值才出发）
起点库存至少 / 终点库存至多	阈值模式的两个触发条件（0-1000000）
策略	平衡交通 / 最短路径 / 优先级通行
优先级	0-9，避让时谁让谁

勾选 启用导航任务 后点 应用导航任务 生效；停用 解除。LIVE
行实时显示相位（前往起点/前往终点/待命…）、决策（正在避让车辆/正在重新规划…）、已完成趟数与里程。

> 关键语义：普通货架必须真的有足量该物品才发车，缺货/不足时车辆原地等待，绝不会凭空生成货物。新放置的车保持空载待命。

8.5 无人机编程

同样两处入口（悬浮窗 / 导航控制页）。与 AGV 的差异：

- 取货点与卸货点都可以是任意楼层的仓储位置（选项带 L{层} 前缀）；
- 采用 26 邻域三维 A*，直接改变 X/Y/Z，不需要升降井；设施上方净空 1.2m、多机间距 3.0m 自动保持；
- LIVE 3D PATH 会画出当前位置→装卸安全悬停点→三维转向节点的路径；若起终点在同一层会提示选择不同楼层；
- 没有策略/优先级项；供货方式与阈值语义与 AGV 相同。

8.6 仓储总览页

08 货物仓储 → 库存控制 页给出全局台账：库存位置表（点行可跳转选中该设备）、运输层汇总、以及按物品的 +产出 -消耗 台账。口径提醒（页面原文）：*仿真启动后，仅从入货仓库实际取出的货物记为消耗，送入出货仓库的货物记为产出。*

9. 运行仿真与监控

9.1 控制入口

- 主控：右侧面板 SimPanel —— 启动/暂停、重置、倍率 $\times 0.35 \times 0.5 \times 1 \times 2$ 、逻辑时间、在途数、利用率、OUTPUT 产出与 INPUT 消耗列表；
- 生产视图（03）顶部有同样的快操按钮；
- 视口底部状态灯：仿真运行中 / 仿真已暂停。

9.2 行为须知

- 仿真引擎使用固定随机种子，同样的布局与操作得到完全相同的运行结果（方便对比方案）；
- 任何结构变更（增删移设备、改配置、加载存档）都会重建引擎并把仿真置为暂停，需要重新点启动；
- 暂停时传送带滚动与货物运动同步冻结，画面静止；
- 大型工厂（>120 对象）快照频率自动降低以保帧率。

9.3 性能建议

- 顶栏  → 渲染目标帧率：一般 60 FPS 足够；高刷屏且工厂不大时可切 120 FPS；
- 不需要的上下文楼层用眼睛开关隐藏；
- 空闲且暂停时渲染自动进入省电的按需模式，属正常现象（一动就恢复）。

9.4 视觉检测 Demo：视频实时 YOLO 识别

这是针对 PCB 等工业制成品的缺陷识别 Demo。它与右侧面板的 OpenCV 手动检测是两条独立路径。

使用步骤

1. 打开独立视觉检测页 inspection.html。从双臂视觉质检单元的设备详情进入，或直接访问 <http://127.0.0.1:5173/inspection.html>。2. 点击左侧 01  DEMO YOLO 质检回放，打开 Demo 窗口。3. 视频会自动播放 PCB 测试画面。播放过程中浏览器定期抓取当前帧，发送给本地 AI 服务进行识别。4. 观察窗口左上角状态：

- YOLOv8 / LIVE INFERENCE：当前帧已由真实 YOLOv8 模型识别，黄色框为本次返回结果；
- YOLOv8 / LOADING MODEL：正在等待模型首次加载；
- YOLOv8 / MODEL OFFLINE：AI 服务或模型不可用，页面不会伪造检测框。

5. 下方“当前识别”“问题累计”和“推理耗时”分别显示最新类别、当前帧检测数量和服务端推理耗时。

当前模型识别：缺孔 missing_hole、鼠咬 mouse_bite、断路 open_circuit、短路 short、毛刺 spur、多余铜 spurious_copper。

启动要求

一键启动会在 <http://127.0.0.1:8000> 启动可选 FastAPI AI 服务。实时 YOLO Demo 需要：

- ai-service/requirements.txt 中的 ultralytics 依赖；
- ai-service/models/pcb_defect_yolov8s.pt 模型文件；
- 浏览器可以访问 AI 服务 8000 端口。

可先检查 <http://127.0.0.1:8000/api/vision/yolo/health>。返回 modelExists: true 但 loaded: false 是正常的懒加载状态，第一次提交视频帧后才会加载模型。

10. AI 诊断页

底部 04 诊断 进入。页面右上角可在两种模式间切换：诊断 · Agent  生成式工厂 · 黛玉规划。

> 所有修改类动作都必须经你人工审批，AI 不能直接改工厂；只读动作随时安全。

10.1 Agent 控制室（黑白工业风）

命令栏（页面上方）：

- 目标输入框（≤2000 字），示例：
- 检查当前工厂的生产、库存和物流瓶颈（默认）
- 吞吐提升到每小时 120 件，场地宽度不超过 30 米

- 模式按钮：☒ 只读分析（不改任何东西）/ ☒ 方案设计（会产出待审批 Patch）；
- 辅助巡检：☒ 自动巡检（跑一轮）、☒ 定时巡检（每 60 秒自动一轮，激活时图标旋转）；
- 主执行按钮：▶ 开始分析（方案设计模式下为 生成方案）。

左栏 执行计划：编译目标约束 → 读取工厂上下文 → 执行只读工具 → 汇总证据与结论（方案设计再加校验变更方案）。工具活动区会列出实际调用的只读工具（读取工厂快照、构建依赖图、诊断瓶颈等 12 个）。

中栏 诊断结果：headline + 置信度 + Finding 卡片（分类 库存/物流/生产/系统，附证据与建议）；卡片上的 © 定位按钮会跳转并选中对应 3D 对象。

右栏 运行证据：当前吞吐/在制品/成品/仿真时间与工厂版本计数；结构变了可点 重新分析当前版本。

10.2 方案审批闭环（Patch）

方案设计完成后出现 方案变更 卡片：

1. 先点 运行双分支对比——系统用副本分别仿真「当前版」和「候选版」，对比吞吐/累计成品/在制品/阻塞/平均运输时间/库存六项指标，给出 建议应用/建议迭代/建议放弃；2. 同意 → ☒ 批准并应用（应用到你的存档，可 ☒ 回滚）；拒绝 → ☒ 拒绝 并填写原因，之后可 ☒ 重新规划；3. 有分析结论但没有 Patch 时，会出现根据当前结论生成修复方案 按钮。

10.3 自动巡检与定时巡检

- 单次 ☒ 自动巡检：后台跑一个 60 秒只读证据副本（不动你的工厂），记录吞吐/利用率/阻塞/在途的指标时序，然后输出中文报告横幅；
- ☒ 定时巡检：每 60 秒自动重复上述过程，直到你再次点击、切换项目或离开诊断页；
- 报告内容：趋势判定（上升/稳定/下降/临界 + 预计耗尽时间）、瓶颈因果链（如「供料者 A → 约束设备 B」）、待机能耗偏高的设备、货架耗尽预警；
- 劣化判定：相比基线吞吐降幅 $\geq 25\%$ 、阻塞增加或交付停滞时，报告会标注劣化并预填修复目标；
- 巡检绝不写入存档；报告的 AI 润色失败时自动回退确定性模板，功能不会因没有 AI 而消失。

10.4 生成式工厂（黛玉规划）

适合「从零造一个新厂」或大改：输入自然语言需求（如“齿轮箱月产 5000 件”），系统确定性解析成约束，搜索多个候选布局并逐一跑副本仿真，最后给出 Top 方案对比；校验通过后可一键应用到当前存档。

已知限制：针对既有产线的局部调整（A-01 场景）有时给不出足够数量的可验证候选而报错，这是当前生成器的已知缺口（详见第 15 节），重试或换更明确的目标通常可行。

11. 存档与数据安全

登录后首次必现 项目弹窗（标题 开始你的工厂；以后从顶栏 项目库 打开）：

- 新建空白工厂：作为未保存草稿进入会话；

- 我的工厂存档：正式存档列表（含更新时间、版本 vN、N 层 N 设施统计）；打开 载入，删除二次确认后不可撤销；
- 自动恢复：置顶的特殊条目，每 60 秒静默写入一份，仅保留最新一份；崩溃/误退后从这里 恢复；
- 文件交换：导入 JSON 把 .forgemind.json 导入为当前会话（不入正式存档库）；导出当前 JSON 下载 {工厂名}.forgemind.json。

顶栏 手动保存存档 才会把当前会话写成正式存档（状态：保存中… / 已保存 / 保存失败 · 重试）。记住：新建与导入都不等于保存。

存档当前为 v6 格式；v1-v5 旧档会自动迁移（含历史乱码楼层名修复），未来版本号会被拒绝以防损坏。

12. 可选 AI 与语音

ForgeMind 默认完全不依赖任何 AI 服务，所有仿真、建造、诊断、巡检都能离线规则运行。AI 只是增强层：

能力	无 AI 时	有 AI 时
Agent 分析/Patch	规则引擎完成	同左（LLM 仅润色表述，不得新增事实）
巡检报告	确定性中文模板	可选润色，失败自动回退模板
语音输入/TTS	按钮置灰，提示 使用 -IncludeAI 启动可选语音服务	ASR + TTS 可用

启用方式：用 scripts\start-forgemind.ps1 启动（默认已含 8000 端口 AI 服务）；远程大模型（DeepSeek 等）只在 ai-service 的服务端环境变量里配密钥，密钥永远不进浏览器和仓库。

13. 快捷键与鼠标速查

按键/操作	生效范围	作用
左键	视口空白	清除选中
左键	建造中	放置设备 / 按住拖绘传送带
右键拖动	任意	平移镜头
右键	建造中	取消当前建造工具
滚轮	任意	缩放
左键拖动	总览/生产	旋转镜头（建造中禁用以防误转）
R	建造中	旋转预览 90°
Esc	全局	关闭工作区/菜单/建造工具；非总览视图退回总览
Delete	多选时	批量删除所选对象

3D 模型查看弹层（3D 观测 按钮）：左键旋转、滚轮缩放、右键平移。

14. 常见问题排查

现象	原因与处理
双击启动后浏览器打不开页面	看 Vite 窗口是否报错；5173 被占用时先跑停止脚本再启动
登录报 认证链路中断	Spring Boot(8080) 未启动；检查 Java 窗口日志，确认 Docker MySQL 已起
建造 ghost 一直是红色	越界、与现有设备重叠或端口格被占；挪开位置或先拆除冲突物
传送带画不出来/断线	拖绘要在地面网格上进行；坡道需两端同向对齐兼容接口
机器放着不干活	①没绑配方（点设备选 RECIPE）②配方被停用③入料口没接到上游④上游无货
传送带有货但下游不吃	下游端口类型不符（蓝入黄出），旋转或改线路
AGV 一直「等待」不发货	起点货架没有足量指定物品，或阈值条件未满足——这是防凭空虚发的保护，不是 bug
无人机没有路径	起终点在同层（本就该走 AGV），或路径被建筑净空挡死；换点重试
定时巡检点了没反应	上一轮还在跑（Worker 串行执行），稍候；离开诊断页会自动取消
YOLO Demo 一直显示离线	检查 AI 服务 8000 端口、/api/vision/yolo/health 和 ai-service/models/pcb_defect_yolov8s.pt；依赖缺失时安装 ai-service/requirements.txt
YOLO Demo 一直显示加载中	首次加载模型需要预热；若超过一段时间仍无结果，检查 AI 服务日志和浏览器是否有多个旧 Demo 标签页同时请求
生成式规划报候选不足	A-01 既有产线调整的已知缺口（见第 15 节）；换目标描述或改用 Agent 方案设计
存档保存失败	后端离线或数据库断开；修复后重点 手动保存存档，期间自动恢复仍在工作
中文显示乱码（旧档）	v6 解析器已自动修复绝大多数历史乱码；若仍遇到请反馈样本

15. 当前已知限制

以下是如实声明的现状，不代表承诺修复时间：

1. 生成式规划的既有产线局部调整（A-01 回归）可能因候选不足 3 个而失败；
2. 浏览器自动化视觉验收受登录墙限制，多数轮次只做了脚本级回归，界面细节以实机为准；
3. 巡检趋势是固定窗口线性外推，不是机器学习预测；纯运行性损耗（如货架随真实时间耗尽）不在逐轮对比中体现；
4. 瓶颈归因为 5 秒采样启发式，单机厂或低负载产线不给约束链；
5. hardware/dm4310-arm/真机机械臂套件的固件未经真机烧录验收，电机量程常量需按批次核对；
6. 部分导入 GLB 的第三方许可证仍在复核中（见《资产与许可审计.md》）。

手册结束。发现手册与实际行为不符时，以代码实测为准并请反馈修正。

16. ForgeLab 开源社区

ForgeLab 是 ForgeMind 的配套开源社区，使用与工作台相同的账号。主门户点击 ForgeLab 后会先进入登录验证页；已有有效工作台会话可以直接验证进入，未登录时使用工作台用户名和密码登录。

社区包含五个板块：

- 工厂存档开源：分享可复现的楼层、设备、仓储和运行结果；
- 模型资源开源：说明 GLB 的原点、包围盒、接口方向、来源与许可；
- 布局经验分享：记录网格、维护通道、坡道落点和避障检查；
- 设计经验分享：沉淀设计目标、约束、判断和验证过程；
- 设计者公告：发布社区规范、版本变化和资源治理说明。

16.1 阅读、下载和讨论

点击帖子卡片会进入独立文章页。正文、作者、版本信息、相关帖子和附件都在文章页展示。后端已接入的附件通过鉴权下载接口读取；仍在公共目录中的模型和视频直接使用资源路径。阅读资源时先确认许可和来源，再下载或复用。

文章底部可以喜欢帖子、跳到现场讨论、喜欢单条回复和发布新回复。回复优先写复现条件、版本、约束和证据，不要只留下无法检查的结论。

16.2 发布帖子

点击社区主视觉的“发布帖子”按钮打开编辑窗口，依次填写板块、标题、摘要和正文；需要时增加图片、工厂存档或资源包，并在 AI 润色提示词中说明希望调整的方向。AI

只帮助整理表达，发布前仍需人工检查事实、版本、许可和附件内容。

登录并且后端可用时，帖子与附件会写入 ForgeLab

数据库；网络暂时不可用时，页面会明确提示并保留当前会话草稿，不会把本地草稿伪装成已同步的公共资源。

16.3 通知与工厂助手

通知中心显示社区公告、回复和点赞事件，可单条标记已读或全部标记已读。右侧 ForgeLab

工厂助手默认使用本地规则回答发布、许可、布局和设计问题；启用可选 AI 服务后，AI

可以进一步润色解释，但不会替代工厂仿真的碰撞、寻路、库存或物料结论。

本手册更新于 2026-08-31。

ForgeMind 已实现功能模块技术文档

项目： ForgeMind 智能工厂数字孪生平台 文档版本： 0.3.0 整理日期： 2026-08-19；2026-08-26 补充修订 适用范围：当前仓库中已经实现、可运行或已经建立接口契约的功能模块

本文档补充 宝钢自研工厂渲染引擎技术文档 和 黛玉智能工厂思考引擎技术文档，重点说明 ForgeMind 除双引擎之外的业务、交互、仿真、数据和 AI 模块。文档中的“已实现”表示代码已经存在并可被当前应用调用；“骨架/占位”表示接口和数据契约已建立，但还没有接入完整生产能力。本版本按 2026-08-19 的代码状态整理，第 1-21 章中的个别细节（如存档版本号、面板按钮名）可能落后于后续迭代——凡与代码不符处，以 当前实现总览 和实际运行结果为准；2026-08 下旬新增的自动巡检子系统与真机硬件套件见第 22 章补充。用户操作视角的完整手册见 ForgeMind-用户使用手册。

1. 功能总览

模块	主要入口	当前状态	核心职责
应用壳与工作区	src/App.tsx	已实现	登录态切换、四种工作视图、全局快捷键、指标栏
生产控制台	src/components/ProductionWorkspace.tsx	已实现	俯视地图、设备登记、物流流向、产出统计和仿真控制
认证与登录演出	src/store/auth.ts、src/scene/LoginCameraRig.tsx	已实现	注册、登录、续登、登出、舱门动画和欢迎音频
工厂状态管理	src/store/forgeMind.ts	已实现	低频编辑状态、选择态、撤销重做、仿真快照
网格建造系统	src/scene/BuildPlacer.tsx、src/game/grid.ts	已实现	放置、碰撞、旋转、拖拽输送带、转角识别
多楼层工厂	src/scene/FactoryFloorSystem.tsx、FloorSwitcher.tsx	已实现	L1/L2/L3 楼层高度、切换和独立产线呈现
设备目录与模型映射	src/game/types.ts、src/scene/EquipmentModel.tsx	已实现	设备规格、足迹、端口、吞吐、资产路径和视觉回退
设备资源导入	src/game/resourcePack.ts、ResourceImportDialog.tsx	已实现	JSON/GLB 选择、拖放、字段校验、模型预览和封面生成
用户资源持久化	src/api/resources.ts、backend/.../ImportedResourceController.java	已实现	资源按用户保存、恢复、模型下载和引用归属校验
物品与配方	src/game/item.ts、ItemPanel.tsx、RecipePanel.tsx	已实现	多输入/多输出配方、加工时长和引用关系维护
离散事件仿真	src/game/simulation.ts、SimulationRunner.tsx	已实现	固定步生产物流、机器状态机、分流汇流、背压
场景和模型系统	src/scene/、public/models/	已实现	高精度工艺设备、Panda URDF、物料可视化、登录舱
仓储与 AGV	src/components/WarehouseWorkspace.tsx、AgyNavigationControl.tsx	已实现	库位、运输层、AGV 任务、路径、避让和实时状态
无人机跨层运输	src/components/DroneNavigationControl.tsx、src/game/droneNavigation.ts	已实现	L1 停靠、升降井、L2/L3 航路和仿真驱动运输
宝钢渲染引擎	src/engine/daiyu/	已实现	精确实例化、预热、运行时审计和 4060 预算策略

模块	主要入口	当前状态	核心职责
黛玉智能工厂思考引擎	src/game/generativeFactory.ts	已实现	需求解析、产线生成、布局调整、路由校验、仿真评估、What-if 和 ROI
本地 JSON 存档	src/game/save.ts	已实现	导出、导入、运行时校验和浏览器下载
Spring Boot 存档/认证后端	backend/	已实现	MySQL/Flyway 持久化、BCrypt 密码、数据库会话 token、按用户隔离工厂和资源
AI 服务	ai-service/main.py	已实现（本地编排）	FastAPI 健康检查、Ollama 助手、NDJSON 流式回复、工具目录、ASR/TTS 网关、PCB YOLOv8 实时帧推理
网页语音助手	src/components/AssistantVoiceButton.tsx、src/game/assistantVoice.ts	已实现（依赖本地服务）	麦克风录音、BT 关键字唤醒、ASR、流式回答和 TTS 播放
独立语音助手	voice-chat/voice_chat.py	独立可运行	不依赖网页的本地 ASR → Ollama → TTS → 播放闭环
仿真回归工具链	scripts/	已实现	闭环、转弯、分流、汇流、背压和 A-01 基地验证

2. 系统总体架构

flowchart LR
 UI[React UI / 工作区] --> Store[Zustand 低频状态]
 UI --> Scene[React Three Fiber 场景]
 Store --> Runner[SimulationRunner]
 Runner --> Sim[SimulationEngine]
 Sim --> Snapshot[10Hz SimulationSnapshot]
 Snapshot --> Store
 Store --> Save[本地 JSON 存档]
 Store --> API[前端 API 客户端]
 API --> Spring[Spring Boot 8080]
 API --> AIService[FastAPI 8000]
 BrowserVoice[网页麦克风 / BT 唤醒] --> AIService
 AIService --> ASR[Paraformer ASR]
 AIService --> Ollama[本地 Ollama LLM]
 AIService --> TTS[BT TTS / Sherpa fallback]
 Voice[独立语音脚本] --> ASR
 UI --> Diagnostics[诊断 / Generative Factory]
 Scene --> Baochai[宝钗渲染引擎]
 Baochai --> GPU[Three.js / WebGL]
 Diagnostics --> Daiyu[黛玉智能工厂思考引擎]
 Daiyu --> Candidate[Layout / Simulation / ROI]
 Candidate --> Store

2.1 分层原则

1. 业务数据层：FactoryObject、Item、Recipe 描述工厂结构和工艺定义。2. 仿真层：SimulationEngine 只处理离散生产逻辑，不依赖 React 或 Three.js。3. 状态协调层：Zustand 保存编辑态和低频仿真快照；高频物料位置不进入响应式状态。4. 宝钗渲染引擎：Three.js/R3F 将对象和快照映射为视觉；宝钗负责批处理、预热和运行时预算。5. 黛玉思考引擎：读取 FactoryState、GenerationSpec 和仿真快照，生成可验证的布局候选、调整方案、诊断解释和经济性对照；它不直接操作 GPU 场景。6.

外部服务层：Spring Boot 负责持久化与认证，FastAPI 负责 AI/ASR/TTS 编排，均不进入实时仿真主循环。

3. 应用壳与工作区模块

3.1 视图模型

src/App.tsx 定义四种工作区视图：

视图	代码	相机	用途
overview	01	等距视角	查看基地布局和设备概况
build	02	俯视	网格放置、旋转和线路编辑
flow	03	生产控制台	俯视地图、设备登记、物流流向和产出统计
diagnostics	04	俯视诊断	查看仿真状态、堵塞和利用率

切换视图时，changeView 会自动退出建造工具，避免隐藏的 ghost 或拖拽状态继续影响场景。顶部导航、左侧工作区导航和视口快速切换 dock 共用同一个 view 状态。

3.2 指标和面板

当前右侧面板和工作区由以下低频组件组成：

- SimPanel：逻辑时间、在途物品、机器平均利用率、播放/暂停、倍率和产出/消耗。
- InfoPanel：选中设备的坐标、朝向、足迹、端口、配方或 source 输出绑定。
- BuildMenu：按采集、加工、装配、物流分类显示设备目录。
- ItemPanel：创建和删除物品类型。
- RecipePanel：创建和删除多输入/多输出配方。
- ProductionWorkspace：在 flow 视图中提供四个标签页：地图全览、设备详情、物流流向和产出统计；地图支持全部/加工设备/物流节点筛选、拖动浏览、节点选择和 在途物料显示。
- AssistantVoiceButton：提供手动录音和 BT 关键字唤醒；浏览器只在本地内存保留短音频窗口并发送到本地 ASR 服务。

生产控制台的设备总数、加工单元、物流节点、产出总量、在途物料和设备利用率来自当前 store/仿真快照。生产效率目前仍显示固定的演示读数（仿真启动后为 92.3%），不能作为真实 OEE 或产线效率结论；可追溯实时量应以 SimulationSnapshot.stats、MachineRuntime.processingTime 和 itemLots 为准。

3.3 全局快捷键

- Esc：退出建造工具，并从非总览视图返回总览。
- R：旋转 ghost 或当前放置工具。
- Ctrl+Z：撤销。
- Ctrl+Shift+Z / Ctrl+Y：重做。
- 设备工作面板：1/2/3 切换分拣、焊接、装配任务；M 切换自动/手动；Space 发送夹爪命令。

输入框、文本域、下拉框和可编辑元素会屏蔽应用级快捷键，避免编辑表单时误操作工厂。

4. 认证与登录演出模块

4.1 认证状态机

src/store/auth.ts 使用三个阶段驱动 UI 和相机：

```
stateDiagram-v2
    [*] --> elevator
    elevator --> entering: login/register 成功
    entering --> factory: 门开 + 推镜完成
    factory --> elevator: logout
    elevator --> factory: 本地 token 续登成功
    entering --> elevator: 用户刷新或会话失败
```

- elevator：只显示电梯舱，LoginCameraRig 独占相机。
- entering：登录成功后打开舱门、播放欢迎音频并推镜进厂。
- factory：挂载完整 UI、OrbitControls 和正常工厂交互。

4.2 前端认证流程

src/api/auth.ts 提供：

方法	路径	作用
register	POST /api/auth/register	注册并返回 token
login	POST /api/auth/login	登录并返回 token
fetchMe	GET /api/auth/me	用 Bearer token 获取当前用户
logout	POST /api/auth/logout	删除服务端数据库会话

请求默认 5 秒超时。成功 token 写入 localStorage['forgemind.token']；应用挂载时调用 restoreSession，续登失败会清理 token 并回到电梯舱。

4.3 舱门与推镜

LoginCameraRig 采用帧循环而非 CSS 动画：

- 舱门打开：1350 ms，smoothstep 缓动。
- 门开确认停顿：360 ms。
- 推镜进入工厂：2100 ms，easeInOut 缓动。
- ElevatorCabin 通过共享的 doorState.t 驱动门叶位移、灯光强度、入口光幕透明度和地板轨道运动。
- 相机越过 $x < -11.92$ 的舱体隐藏阈值后立即隐藏电梯舱，避免进入工厂后继续渲染舱体。

登录成功后 App.tsx 播放 /audio/welcome_home_bt.wav。音频播放失败不会阻塞入场动画。

4.4 后端安全边界

Spring Boot 当前使用 BCrypt 保存密码哈希；会话 token 是 UUID，服务端只把 token 的 SHA-256 摘要保存到 MySQL，默认有效期 30 天。因此：

- 密码不会明文落盘。
- 后端重启后 token 仍可持续用，过期会话会在校验时清理。
- 当前没有完整 Spring Security 过滤链、刷新 token、角色权限和限流。
- @CrossOrigin(origins = "**") 适合本地演示，不适合生产部署。

5. 工厂状态管理模块

5.1 Zustand 状态边界

src/store/forgeMind.ts 是编辑器和 UI 的低频状态中心。主要字段包括：

```
objects: FactoryObject[]
items: Item[]
recipes: Recipe[]
selectedId: string | null
buildType: BuildType | null
ghost: Ghost
ghostPath: GridPos[]
simSnapshot: SimulationSnapshot
simPlaying: boolean
simSpeed: number
```

设计约束是：对象布局、绑定关系和 UI 状态进入 store；每一帧的传送带 offset、机器骨骼和渲染矩阵不进入 store。

5.2 编辑历史

编辑器维护最多 80 个 FactoryHistoryEntry，每条记录包含 objects 和 selectedId。放置、删除、旋转、绑定配方和绑定物品会进入历史栈；撤销/重做同时递增 simResetTick，让 SimulationRunner 重新初始化仿真。

导入存档和清空工厂会清空历史栈，避免跨存档撤销产生对象引用错乱。

5.3 引用一致性

删除物品时，store

会同步删除所有配方中引用该物品的输入/输出端口，并移除完全没有端口的配方，避免仿真读取悬空 itemId。删除配方不会自动清理机器上的 recipeId，因此后续可增加绑定引用检查或自动解绑。

6. 网格建造与线路编辑模块

6.1 坐标和足迹

src/game/grid.ts 规定：

- 1 格 = 1 米。
- GridPos 的 x/z 是对象足迹的最小角锚点。
- gridToWorld 将格子中心转换为 Three.js 世界坐标。
- 对象视觉锚点使用旋转后足迹的几何中心。
- rotation 只允许 0/90/180/270 四个方向。

- 建造边界为 `BUILD_BOUND = 24`。

`rotatedFootprint` 在 $90^\circ/270^\circ$ 时交换宽深；`occupiedCells` 枚举旋转后的所有占用格。

6.2 放置合法性

`canPlace` 按以下顺序判断：

1. 计算旋转后的足迹。
2. 检查足迹是否越出 `[-24, 24]` 建造边界。
3. 枚举候选对象和已有对象的占用格。
4. 使用格子集合判断是否重叠。

因此碰撞是离散格碰撞，不依赖 `Three.js` 包围盒，也不会因模型视觉细节或阴影边界造成误判。

6.3 指针、ghost 和旋转

`BuildPlacer` 使用 `Three.js Raycaster` 将屏幕指针投射到 $y=0$ 平面，再通过 `Math.floor` 转成网格坐标。移动时更新 `ghost` 的位置和合法性；`GhostPreview` 根据 `ghost.valid` 显示合法/非法状态。

按 `R` 后旋转 90° ，旋转结果会立即重新经过 `canPlace` 校验。左键确认放置，空工具状态下点击地面会清除选择。

6.4 输送带连续拖拽和转角

右键会把当前位置写入拖拽锚点，随后 `buildPath` 将多个锚点和当前指针拼成 Manhattan 路径：先沿 `X`，再沿 `Z`。`pathRotations` 根据相邻格计算每段方向，逐段调用 `placeAt`。

线路转角由 `getConveyorLinks` 根据上游/下游连接方向识别：

- 直线段使用精确滚筒模型和方向箭头。
- 90° 转角使用专用程序化 `ConveyorCorner`，避免模型朝向和实际物流方向不一致。
- 转角不使用普通直线滚筒批次，保证视觉和仿真端口语义一致。

6.5 端口语义

`objectPortCells` 为每类对象计算外部输入/输出格：

- 普通设备：后侧输入、前侧输出。
- `splitter`：后侧输入，前/左/右三路输出。
- `merger`：后/左/右三路输入，前侧输出。
- `assembler`：三个独立 line-side dock，允许不同物料从不同方向进入。
- `source`： 3×2 复合来料站只使用近中心实体皮带所在的单条输出通道。
- `conveyor`：输入允许后/左/右，输出保持单一方向，支持 Manhattan 转角线路。

真实连接不仅要求端口格相邻，还要求上游输出端和下游输入端重合；仿真和视觉端口标记共用这套计算。

7. 设备目录、资产映射与模型回退

7.1 设备定义

`src/game/types.ts` 中的 `OBJECT_DEFS` 是设备目录唯一来源，包含：

- 业务类型和角色。

- 中文名称、英文副标题、功能描述。
- 模型或资产路径。
- 足迹、颜色、强调色、高度。
- 吞吐、能耗、输入、输出和端口定义。

当前设备类型包括原料源、原料仓储架、滚筒输送线、通用工作站、数控中心、液压冲压、机器人装配、视觉检测、清洗去毛刺、AGV、成品缓存、分流器和汇流器。

7.2 资产来源分类

BUILD_ASSET_PATHS 将设备划分为：

- 中心拆分资产：从中心工业模型中抽取的通用工作站、输送段、数控中心、Panda 装配单元和检测模块。
- 独立高精度工艺资产：冲压、清洗、仓储、流节点等独立 GLB。
- 程序化结构：没有对应外部节点的简单货架、基础底座、箭头和状态指示。

EquipmentModel 为每类设备提供 Suspense 回退。GLB

加载失败时显示程序化设备，而不是让整个场景挂起或出现空白。

7.3 精度保护

运行时优化共享几何、材质和实例矩阵，不修改 public/models 中的原始 GLB、DAE 和

URDF。模型精度、业务足迹和端口作用分开管理：模型负责视觉，OBJECT_DEFS/grid.ts 负责业务碰撞和连接。

8. 物品与配方模块

8.1 数据模型

src/game/item.ts 定义：

```
type ItemCategory = 'raw' | 'intermediate' | 'product'
```

```
interface Item {
  id: string
  name: string
  category: ItemCategory
  color: string
  size: number
  note?: string
}
```

```
interface Recipe {
  id: string
  name: string
  inputs: RecipePort[]
  outputs: RecipePort[]
  durationSec: number
}
```

Item 是类型定义；仿真中的在途实体是 ItemLot，二者不能混淆。

8.2 默认工业词汇

系统预置钢制毛坯、冷轧钢板、铜线盘、标准紧固件、机加工壳体、冲压壳体、洁净零件、电机总成和已检电机等物品，并提供机加工、冲压、线圈绕制、紧固件齐套、清洗、电机装配、视觉终检和包装配方。

8.3 配方编辑器

RecipePanel 支持：

- 选择输入物品和数量。
- 选择输出物品和数量。
- 设定加工时长。
- 在提交前删除端口行。
- 在列表中删除配方。

提交条件是名称非空、至少一个输入和至少一个输出；加工时长在 store 中被限制为不小于 0.1 秒。

8.4 设备绑定

InfoPanel 为机器提供配方下拉框，为 source 提供输出物品下拉框。绑定结果写回 FactoryObject.recipeId 或 FactoryObject.itemId，随后触发仿真重建，保证运行时使用最新配置。

9. 离散生产仿真模块

9.1 运行原则

src/game/simulation.ts 是纯 TypeScript 仿真内核，也是生产逻辑唯一真相源。它不导入 React、Three.js 或 DOM，可在浏览器、Node 脚本和未来后端引擎中复用。

固定参数：

参数	值	含义
SIM_STEP	0.05 s	固定逻辑步长，20 Hz
CONVEYOR_SPEED	2 格/s	传送带离散槽位推进速度
SOURCE_INTERVAL	1.0 s	source 产出计时
SOURCE_TRANSFER_TIME	1.2 s	source 拾取/放置动画对应的逻辑时长
LOAD_TIME	0.5 s	机器收料过渡
OUTPUT_TIME	0.3 s	机器出料过渡

引擎使用累加器吸收真实时间，将每次 advance(dtSec) 拆成多个固定 50 ms 步；最多执行 1000 步，避免浏览器长时间冻结后出现无限追赶。

9.2 Source 生命周期

Source 状态为 idle、picking、placing、blocked：

1. 没有绑定 itemId 或没有下游时进入 blocked。
2. 达到产出间隔后开始 transfer。
3. 前 52% 进入 picking，后续进入 placing。
4. 到达 100% 时尝试把物品放入下游传送带或机器输入缓冲。
5. 下游满时保持 blocked，物品不会凭空生成。

9.3 传送带和在途物品

每个传送带运行时容量为 1 个 ItemLot:

```
interface ItemLot {  
  id: string  
  itemId: string  
  conveyorId: string  
  offset: number // 0..1, 沿当前段的进度  
}
```

物品每个固定步增加 $CONVEYOR_SPEED * dt$ 。到达 $offset \geq 1$ 后按输出端口尝试进入下游:

- 空的传送带: 转移到下一段并重置 offset。
- 可接收的机器: 进入机器输入缓冲。
- 空格: 作为工厂出口, 物品离开场景。
- 下游满或端口不连接: offset 固定为 1, 形成头堵。

传送带按对象 ID 排序后推进, 保证同一布局 and 同一种子下的结果稳定。

9.4 机器状态机

机器状态为 idle → loading → processing → output:

- idle: 检查配方所有输入是否满足数量。
- loading: 消耗输入并执行收料过渡。
- processing: 按配方时长推进, 并累加 processingTime。
- output: 尝试把第一个输出端口的产品送入下游; 下游满时停留在 output。

SimulationSnapshot 对 UI 暴露机器状态、进度、输入缓冲、累计加工时间、在途物品和产出/消耗统计。

当前数据结构支持多输出配方统计, 但机器路由仍按 `recipe.outputs[0]` 放入下游, 这是 MVP 的明确边界。

9.5 分流、汇流和连接校验

- splitter 使用 branchCursor 轮询三条输出, 优先选择当前游标后第一个可接收分支。
- merger 使用后/左/右三个输入端口, 统一从前侧输出。
- isConnected 要求上游占用格和下游输入端口格相交; 不满足时不允许传输。
- 大尺寸设备的中心双通道和装配单元的多 dock 均由 objectPortCells 统一处理。

9.6 仿真驱动器

SimulationRunner.tsx 在 App 顶层挂载一个模块级 SimulationEngine 单例:

- 固定种子 20260813。
- requestAnimationFrame 获取真实时间。
- 根据 simPlaying 和 simSpeed 推进引擎。
- 仅每秒 10 次把 getSnapshot() 写入 Zustand。

- 对象、配方变化或 simResetTick 变化时重建引擎。

这样 React 只处理低频快照，Three.js 可以继续以每帧频率渲染物料位置和机械动作。

10. 场景、物料与机器人模块

10.1 场景组织

FactoryCanvas 负责 Canvas、相机、灯光、雾、网格地面、场景预热、登录舱和相机控制；FactoryScene 负责对象分组和渲染批次选择。

对象按类型拆分为传送带、通用机器、AGV、冲压、清洗、仓储、source、inspection 和 Panda 批次。动态对象仍可回退到 FactoryObjectMesh 独立渲染。

10.2 物料视觉

ItemLotMesh 将仿真快照中的 ItemLot

映射到当前传送带对象的位置和方向，显示为轻量占位物料。物料位置来自仿真状态，不在渲染层自行决定物流结果。

10.3 Panda 机械臂

PandaArmModel 使用 /models/panda/panda.urdf 和 DAE visual link：

- 模板 Promise 单例加载，避免每台机械臂重复解析 URDF。
- 加载完成后等待视觉 Mesh 和有限包围盒，再进行 Z-up 到 Y-up、归一化和底面对齐。
- 静止机械臂进入 DaiyuPandaBatch 精确批次。
- source 拾取/放置或装配工作时，恢复独立 URDF 关节树。
- 自动动作使用低迭代 Damped Least Squares IK；手动模式支持键盘和 Gamepad。
- 通过 forgemind:robot-command 自定义事件接收任务、模式和夹爪命令。

10.4 登录舱和正式工厂相机

登录阶段不挂载 OrbitControls，避免舱门推镜与用户控制抢夺相机。正式工厂阶段根据视图选择相机预设，并以 960–1380 ms 的过渡时间插值到目标位置。建造模式暂时禁用 OrbitControls，避免拖拽线路时同时旋转视角。

宝钗的批处理、预热、阴影和性能数据详见独立文档，不在本篇重复展开；黛玉的生成、调整和评估链路见独立思考引擎文档。

11. 存档、导入导出与后端同步

11.1 本地存档格式

FactorySave 当前版本为 6（2026-08-25 时点；版本随能力演进递增，最新事实以总览为准）：

```
interface FactorySave {  
  version: number  
  savedAt?: string  
  objects: FactoryObject[]  
  items: Item[]  
  recipes: Recipe[]  
}
```

serializeSave 生成格式化 JSON；导出文件名为 {工厂名}.forgemind.json；导入通过项目弹窗的「导入 JSON」进入当前会话。

11.2 运行时校验

parseSave 会检查：

- 顶层对象是否有效。
- objects/items/recipes 是否为数组。
- 位置、旋转、类别、端口数量和物品引用是否合法。
- 配方端口引用的 itemId 是否存在。

未知字段会被忽略，关键字段错误会抛出中文错误，LeftPanel 将错误显示给用户。

11.3 存档版本与兼容性

当前存档版本为 v6。解析器接受 v1-v5 并逐级迁移到

v6（含楼层名称、仓储实例命名、车辆任务等历史字段迁入；Windows-1252 误解码的历史中文会自动恢复，无法恢复的连续问号回退内置目录名称或业务 ID），同时拒绝缺失、非法或未来版本；对象类型校验直接复用完整 BuildType 目录，因此 A-01 的全量设备可以导出后再次导入。对象、物品和配方 ID 也会检查重复，避免导入后出现引用歧义。

11.4 Spring Boot 同步

登录态下通过顶栏「手动保存存档」写入正式存档、项目弹窗管理存档库（打开/删除/自动恢复），另有每 60 秒静默写一份自动恢复；src/game/api.ts 提供带超时的远端客户端：

方法	路径	作用
isBackendOnline	GET /api/factory/health	探测后端
fetchRemoteSave	GET /api/factory	拉取整厂存档
pushRemoteSave	PUT /api/factory	覆盖远端存档

后端离线不会阻塞本地编辑和本地 JSON 存档。

12. Spring Boot 后端模块

12.1 技术栈和持久化

- Java 17。
- Spring Boot 3.3.5。
- spring-boot-starter-web。
- spring-boot-starter-jdbc + MySQL Connector/J。
- Flyway 数据库迁移。
- spring-security-crypto，只用于 BCrypt。

- MySQL 数据库 forgemind 保存用户、会话、工厂、楼层、物品、配方、设备布局 and 预留快照。

数据库初始化脚本位于 backend/src/main/resources/db/migration/；当前已执行 v1-v4 迁移。高频 ItemLot 和机器运行态仍由仿真引擎持有，不写入实时 CRUD 表。设备绑定 ID 在后端按同一工厂范围校验，避免复合主键外键无法正确表达可空绑定的问题。

12.2 REST 接口

认证接口：

方法	路径	说明
POST	/api/auth/register	用户名至少 2 个字符，密码至少 6 位
POST	/api/auth/login	BCrypt 校验后签发 UUID token
GET	/api/auth/me	Authorization: Bearer <token>
POST	/api/auth/logout	删除数据库会话

工厂接口：

方法	路径	说明
GET	/api/factory	返回完整 FactorySave
PUT	/api/factory	覆盖并返回完整 FactorySave
GET	/api/factory/health	服务健康检查

后端只负责静态结构 CRUD，不推进前端实时仿真。未来如果迁移到 Java 仿真引擎，应先定义版本化事件协议，不能直接让 CRUD 接口修改运行时状态。

13. AI 服务模块

13.1 当前实现

ai-service/main.py 是 FastAPI 服务，监听默认 8000 端口，启用全域 CORS。当前接口：

方法	路径	返回
GET	/api/ai/health	{"status": "ok", "service": "forgemind-ai"}
GET	/api/ai/tools	版本化工具目录
POST	/api/ai/assistant	调用本地 Ollama，返回 answer/source/note/action 结构化响应
POST	/api/ai/assistant/stream	返回 NDJSON 增量文本和最终动作信封
POST	/api/ai/asr	Paraformer WAV 语音识别
POST	/api/ai/tts	BT TTS 代理，失败时回退 Sherpa VITS
GET	/api/vision/yolo/health	PCB YOLOv8 模型文件、加载状态和类别列表
POST	/api/vision/yolo/detect	接收一帧 JPEG/PNG base64，返回真实 PCB 缺陷框、类别、置信度和推理耗时

请求模型：

```
{
  "question": "怎么提高产量？",
  "context": {
    "timeSec": 120,
    "bottlenecks": []
  }
}
```

助手默认使用服务端规则模式，不要求 Ollama 或本地大语言模型；只有显式配置远程 provider（当前支持 DeepSeek）时才请求远程模型。可返回普通文本或 1.0.0 工具动作；服务不可用时返回 fallback。服务端会对动作名称、协议版本、参数和上下文对象做基础校验，前端 assistantProtocol.ts 会再次校验并负责执行，因此 FastAPI 不直接改变工厂对象或仿真状态。前端客户端为 src/game/api.ts，支持 NDJSON 流式文本；assistantRuntime.ts 会按短句切分回复并并行请求 TTS，按顺序播放。

当前动作目录位于 contracts/forgemind-assistant-tools.json，共 8

个动作：查询工厂状态、读取对象、选择对象、启停仿真、设置仿真倍率、重置仿真、修改机器配方和绑定 source 物品。查询/定位/仿真控制可直接执行；重置、改配方和改 source 绑定被标记为需要确认的高风险或配置变更动作。

13.2 PCB 视频实时 YOLO 检测

视觉检测 Demo 位于 src/demos/InspectionDemo.tsx，视频资源是 public/videos/inspection-pcb-demo.mp4，模型权重是 ai-service/models/pcb_defect_yolov8s.pt。视频仅提供未叠加预测标签的 PCB 测试画面，检测结果在运行时生成。

```
video.current frame
→ offscreen canvas.toDataURL("image/jpeg")
→ POST /api/vision/yolo/detect
→ Ultralytics YOLO(modelPath).predict(image, conf, imgsz=640)
→ {className, confidence, x1, y1, x2, y2}
→ Canvas overlay
```

前端在播放事件中触发抽帧，并以约 360ms

为最小提交间隔；同一标签页在上一帧未完成时不再发送下一帧。服务端模型采用懒加载，/api/vision/yolo/health 不因健康检查而加载权重；推理过程有单帧并发锁，繁忙时返回 YOLO 正在处理上一帧，请稍后重试。

响应中的 width/height

是原始解码帧尺寸，坐标以该尺寸为基准；前端根据视频元素显示尺寸换算后绘制。模型类别固定为 missing_hole、mouse_bite、open_circuit、short、spur、spurious_copper。

运行时状态必须区分：

- LOADING MODEL：Demo 已打开但尚未得到模型响应；
- LIVE INFERENCE：最近一帧由 YOLO 服务成功返回；
- MODEL OFFLINE：服务请求失败、模型依赖缺失或模型文件不可用。

实时 YOLO Demo 不进入 SimulationRunner，不修改

FactorySave，也不把识别结果写入生产/仓储台账。模型服务是可选依赖；AI 不可用时，工厂建造和确定性仿真不受影响。

13.3 设计边界

AI 只能输出动作目录中的结构化请求，不能直接自由修改工厂。当前已完成协议版本、白名单、参数、对象角色、物品/配方引用和确认门控；副本仿真、产能差异报告和“优化建议先验证再写回”仍属于后续能力。

AI 服务不得进入 SimulationRunner 的每帧或每个固定步，否则会把 LLM 延迟引入实时链路。

14. 网页与离线语音助手模块

14.1 处理链路

网页端 AssistantVoiceButton 已接入主工作区，支持手动录音和默认开启的 BT 关键字唤醒。浏览器端负责采集单声道音频、重采样到 16 kHz、调用本地 ASR，并将识别文本交给统一的 assistantRuntime；回答以流式文本进入短句 TTS 队列，BT TTS 不可用时由 FastAPI 回退 Sherpa VITS。关键字监听只发送有明显能量的短窗口，不上传第三方服务。

网页处理链路：

```
sequenceDiagram
    participant U as 用户
    participant B as 浏览器麦克风
    participant A as FastAPI / ASR
    participant L as Ollama
    participant T as BT TTS
    participant F as Sherpa VITS
    participant UI as ForgeMind UI
    U->>B: 手动录音或说 BT
    B->>A: 16kHz mono WAV
    A->>B: 识别文本
    B->>A: 问题 + 工厂上下文
    A->>L: 工具调用/流式回答
    L->>B: NDJSON 文本 + 动作信封
    B->>UI: 前端校验、执行或请求确认
    B->>T: 按短句合成
    T->>B: WAV
    B->>F: BT 不可用时回退
```

独立模式仍由 voice-chat/voice_chat.py 提供，不共享网页会话：

```
sequenceDiagram
    participant U as 用户
    participant Mic as 麦克风
    participant ASR as sherpa Paraformer
    participant O as Ollama qwen2.5:7b
    participant T as BT TTS HTTP
    participant F as sherpa VITS fallback
    participant Spk as 扬声器
    U->>Mic: 按回车开始/结束
    Mic->>ASR: 16kHz float32 单声道
    ASR->>O: 中文文本 + 对话历史
    O->>T: BT-T274 风格文本
    T->>Spk: WAV 音频
    T->>F: 服务不可用时回退
    F->>Spk: 本地合成音频
```

14.2 模型和运行约束

- ASR: Paraformer int8 ONNX, 4 线程。
- TTS fallback: Sherpa-ONNX VITS 中文模型, 4 线程。
- LLM: <http://localhost:11434/api/chat>, 默认 qwen2.5:7b。
- BT TTS: <http://127.0.0.1:8001/tts>, 不可用时回退本地 VITS。
- 输入小于 0.3 秒或低于静音阈值 0.01 时丢弃。
- 识别到“退出”或“再见”时结束循环。

网页语音入口已能通过统一动作协议驱动当前支持的仿真控制和配置动作，但仍受协议白名单、对象引用校验和用户确认限制；独立 voice-chat 程序仍只提供对话播报，不直接向网页工厂发送动作。任何新语音动作都必须先加入协议目录和前端执行器，不能让语音文本直接执行任意设备操作。

15. 测试与验证工具链

15.1 npm 脚本

```
npm run build
npm run sim
npm run sim:smoke
npm run sim:regression
npm run sim:backpressure
```

npm run build 执行 TypeScript 项目构建和 Vite 生产打包。scripts/run-sim.mjs 使用 esbuild 将 TypeScript 测试脚本临时 bundle 成 Node 可执行模块，结束后清理临时目录。

15.2 回归覆盖范围

scripts/sim-regression.ts 当前覆盖 7 类场景：

1. 基础 Source → 传送带 → 机器 → 出口闭环。
2. 90° 转弯线路。
3. 三向分流器轮询。
4. 双输入汇流器和多输入配方。
5. 下游拒收时的头堵与背压。
6. 3×2 大型设备的中间入口/出口通道。
7. Source picking → placing → grid lot 生命周期。

base-a01-check.ts 额外检查 A-01 预置布局的边界和碰撞、设备类型完整性，并运行 300 秒仿真确认电机装配、终检和包装链路闭合。

15.3 性能验证入口

宝钗开发诊断参数（参数名保留 daiyu 历史标识）：

- ?daiyuStress=300：生成最多 600 个不写入 store 的压力对象。
- ?daiyuDpr=1.86：固定开发环境渲染 DPR，用于近 1080p 采样。

性能指标、P95 帧时、场景审计和 4060 Laptop 实测结果统一记录在 宝钗渲染引擎文档 中。

16. 端到端数据流和不变量

16.1 建造到仿真

```
sequenceDiagram
    participant P as Pointer/R 键
```

```
participant B as BuildPlacer
participant S as ForgeMind Store
participant R as SimulationRunner
participant E as SimulationEngine
participant V as FactoryScene
P->>B: 网格指针/旋转/确认
B->>S: updateGhost/placeAt
S->>R: objects 或 simResetTick 变化
R->>E: init(objects, recipes)
R->>E: 固定步 advance
E->>R: SimulationSnapshot
R->>S: 10Hz 快照
S->>V: 机器状态、source 状态、ItemLot
V->>V: 每帧插值和批处理渲染
```

16.2 必须保持的不变量

- rotation 只取四个方向，且同时决定视觉朝向和物流方向。
- 任何对象不能越出建造边界或与其他对象占用格重叠。
- 传送带满时只能头堵，不能穿透、丢失或凭空复制物料。
- 机器只有在输入数量齐备且配方存在时才能进入 loading。
- 渲染层不能替代仿真层决定物料去向。
- AI、网络请求和语音推理不能进入实时固定步循环。
- 原始模型资产不因性能优化而被隐式减面、替换或改变作用。

17. 启动和排障手册

17.1 仅运行前端

```
npm install
npm run dev
```

访问 <http://localhost:5173>。没有后端时可以继续使用建造、物品、配方、仿真和本地 JSON 存档；登录接口会显示后端连接错误。

17.2 启动 MySQL 与 Spring Boot

```
docker compose up -d mysql
cd backend
mvn package
java -jar target/forgemind-backend-0.1.0.jar
```

默认端口 8080。检查 <http://localhost:8080/api/factory/health>。

17.3 启动 AI 服务

```
cd ai-service
py -3.10 -m venv .venv
.venv/Scripts/pip install -r requirements.txt
.venv/Scripts/python -m uvicorn main:app --port 8000
```

检查 `http://localhost:8000/api/ai/health`。此接口正常只代表 FastAPI 在线；要得到 `source=llm` 的真实回答，还需要 Ollama 和 `qwen2.5:7b` 已启动。BT TTS 不可用时，服务会尝试使用本地 Sherpa VITS 备用模型。

17.4 典型问题定位

现象	优先检查
登录显示 Failed to fetch	Spring Boot 是否监听 8080，浏览器是否允许本地跨源请求
物料不动	source 是否绑定物品、机器是否绑定配方、端口方向是否连接
传送带末端停住	下游是否满、是否没有配方、是否形成预期背压
机械臂不显示	Panda URDF/DAE 路径、模型请求、Daiyu 批次是否接管、浏览器控制台错误
导入存档失败	JSON 版本、对象类型白名单、配方引用的物品是否存在
舱门动画卡顿	宝钗预热状态、Panda 是否在隐藏祖先下跳过 IK、DPR 和阴影预算

18. 多楼层、仓储与跨层物流补充

18.1 楼层模型

FactoryFloorSystem 根据 `floorId` 把对象放到对应的世界高度，FloorSwitcher 只切换可见楼层和当前编辑目标，不复制或重建工厂对象。L1 是仓储和无人机停靠层；L2 是加工/冲压/绕线层；L3 是多输入装配、视觉质检、包装和成品缓冲层。默认楼层定义和完整基础布局位于 `src/game/baseA01.ts`。

18.2 无人机运输

`src/game/droneNavigation.ts` 固定 L1 停靠位、升降井、目标楼层高程、外围高位环线 and 楼层输入点。无人机导航控制页可在仓储工作区打开，选择 L2/L3 目标层查看航路。仿真运行后，SimulationRunner 通过快照驱动三维无人机状态；暂停时不推进运输任务。

18.3 标签与面板可见性

设备、车辆和仓储区域标签属于场景空间内容，不应穿透工作区面板。三维标签和 HUD 面板分别处于不同渲染层；排查标签漏到其他界面时，优先检查场景 overlay 的挂载范围、当前 floor 可见性和面板的 `z-index/portal` 容器。

19. 资源导入与用户隔离补充

19.1 前端流程

BuildMenu 打开 ResourceImportDialog。对话框支持拖放或文件选择，读取资源 JSON 和 GLB，使用 `parseForgeMindProject` 做字段校验，使用 `Model3DViewer / ImportedFactoryModel` 预览模型，并生成设备卡片封面。解析成功后资源先进入 Zustand，用户可以立即在建造目录中使用。

19.2 后端流程

登录态下由 `src/api/resources.ts` 调用 `/api/resources` 保存资源。Spring Boot 的 `ImportedResourceController` 和 `ImportedResourceDbStore` 把元数据、项目 JSON、GLB 文件名和二进制写入 `imported_resource`。App.tsx 在恢复工厂阶段加载当前用户资源并重新建立模型 Blob URL；切换用户时 `forgeMind` 会清理旧资源和 URL。

19.3 安全不变量

- 资源列表、GLB 下载和工厂保存都使用 bearer token；
- factory_object 只有在 resourceId 属于当前用户时才能保存；
- 未登录或后端不可用时，导入可以用于本地预览，但不能被标记为已云端持久化；
- 新增资源类型时，同时检查前端 OBJECT_DEFS、资源包解析器、Spring Boot 对象白名单和迁移/回归样例。

20. 当前边界与后续路线

已知边界

- MySQL 使用 Docker 持久卷；删除 forgemind_mysql_data 会清空开发数据。
- AI FastAPI 已接入本地 Ollama 和动作协议；Ollama 不可用时仍会返回 fallback，不能把 fallback 当作真实模型结果。
- 网页语音入口已经接入工作区，但依赖浏览器麦克风权限、Paraformer、Ollama 和 TTS 服务；独立 voice-chat 仍不共享网页会话。
- 本地和后端存档校验均覆盖当前完整设备目录；新增设备类型时仍需同步更新前后端白名单和回归样例。
- 机器多输出配方的数据结构已支持，但运行时下游路由仍使用第一个输出。
- A-01 中部分 KPI，以及生产控制台的固定“生产效率 92.3%”，是演示读数，不等同于仿真统计。
- 4060 Laptop 与 5 GiB LLM 同时运行的长时间稳定性仍需实机认证。

推荐演进顺序

1. 补充工厂成员邀请、角色权限和多工厂选择 API。2. 为 AI 增加副本仿真、差异报告和优化建议确认流程；当前 action schema、双层校验和确认门控已完成。3. 让独立语音助手复用网页端的版本化安全命令协议，并增加会话/权限边界。4. 增加自动化浏览器验收、显存采样、温度采样和 LLM 并行压力测试。5. 在模型资产不变的前提下评估 HLOD、WebGPU 和离线纹理压缩。

21. 代码索引

路径	说明
src/App.tsx	应用壳、视图切换、KPI、快捷键和登录音频
src/components/ProductionWorkspace.tsx	生产控制台的地图、设备登记、物流和产出标签页
src/components/AssistantVoiceButton.tsx	网页端手动录音和 BT 关键字唤醒入口
src/store/forgeMind.ts	编辑器、历史栈、物品/配方、仿真快照
src/store/auth.ts	认证阶段和本地 token 续登
src/game/types.ts	设备目录和 FactoryObject 类型
src/game/grid.ts	足迹、碰撞、端口和世界坐标
src/game/item.ts	Item、Recipe 和默认工业词汇

路径	说明
src/game/simulation.ts	固定步长离散仿真唯一真相源
src/game/SimulationRunner.tsx	rAF 驱动器和 10Hz 快照桥接
src/game/save.ts	本地存档序列化、解析和下载
src/game/api.ts	Spring Boot 存档和 FastAPI AI 客户端
src/api/resources.ts	用户资源保存、列表恢复和鉴权模型下载
src/game/resourcePack.ts	资源包解析、校验、规范化和封面输入
src/components/ResourceImportDialog.tsx	建造页资源导入窗口
src/components/WarehouseWorkspace.tsx	仓储控制、运输层和物料台账
src/components/AgvNavigationControl.tsx	AGV 导航任务和路径控制
src/components/DroneNavigationControl.tsx	无人机跨层航路控制
src/game/droneNavigation.ts	无人机路线、楼层高程和配送点
src/scene/FactoryFloorSystem.tsx	三维楼层容器和可见性
src/game/assistantProtocol.ts	智能管家 1.0.0 动作目录、上下文和前端校验
src/game/assistantExecutor.ts	智能管家动作执行、确认门控和仿真控制桥接
src/game/assistantVoice.ts	浏览器录音、WAV 编码、ASR 和关键字监听
src/game/assistantRuntime.ts	流式回答、短句 TTS 队列和动作执行桥接
src/scene/BuildPlacer.tsx	射线拾取、ghost、拖拽线路和快捷键
src/scene/FactoryCanvas.tsx	Canvas、相机、场景分组和宝钗接入
src/scene/FactoryObjectMesh.tsx	单对象渲染、端口标记和转角视觉
src/scene/EquipmentModel.tsx	设备模型映射、加载和程序化回退
src/scene/PandaArmModel.tsx	Panda URDF、IK、自动/手动控制
backend/src/main/java/com/forgemind/web/	认证和工厂 REST 控制器
backend/src/main/java/com/forgemind/web/ImportedResourceController.java	用户资源 REST 控制器
backend/src/main/java/com/forgemind/repository/ImportedResourceDbStore.java	资源元数据和 GLB 持久化
backend/src/main/resources/db/migration/V5__create_user_imported_resources.sql	用户资源表
backend/src/main/resources/db/migration/V6__add_resource_reference_to_factory_objects.sql	工厂设备资源引用
ai-service/main.py	FastAPI AI/ASR/TTS 编排服务
voice-chat/voice_chat.py	本地 ASR/LLM/TTS 语音闭环
scripts/sim-regression.ts	仿真回归测试
scripts/base-a01-check.ts	A-01 布局和 300 秒生产闭环检查

22. 补充（2026-08-25）：自动巡检全链路真机硬件套件

本章补充第 1-21 章成稿之后落地的能力，均为当前代码事实。

22.1 自动巡检子系统（诊断页）

围绕「只读证据 → 趋势预警 → 劣化判定 → 受控修复」的完整链路，全部重计算在 Web Worker 中执行：

模块	文件	职责
指标时序	src/game/metricsHistory.ts	36 容量环形缓冲，存吞吐/利用率/阻塞/在途
巡检引擎	src/game/factoryAutopilot.ts	固定种子 60 秒只读证据副本、劣化判定（吞吐降幅 ≥25%、阻塞增量 ≥1、交付停滞）、结构版本基线对比，产出标准 autopilot_* Finding 注入既有分析结果
瓶颈归因	src/game/bottleneckAnalysis.ts	5 秒采样机器状态，按「忙碌×0.7 + (1-相对完成率) ×0.3」排序输出「供料者[ID] → 约束设备[ID]」因果链；导出 runSampledEvidence 供能耗归因复用采样副本
能耗归因	src/game/energyAnalysis.ts	解析设备定义功率字段（如 "22 kW"），运行态满功率、待机按 15% 计，窗口累加能量并给出待机浪费 Top 设备；仅归因提示，不触发修改
库存预测	src/game/inventoryForecast.ts	基于时序的最小二乘斜率外推货架耗尽 ETA（rising/stable/declining/critical 四态）
报告叙述	src/game/patrolNarrative.ts	确定性中文模板逐字节可复现；注入式 narrator 接可选 AI 润色，超时/失败/空回复回退模板且不新增事实
语音播报	src/game/patrolVoice.ts	把报告压缩为一句可朗读警报；优先 AI 服务 /api/ai/tts，失败回退浏览器 speechSynthesis，再失败静默跳过；同内容去重由调用方保存上次文本
Worker 隔离	src/game/autopilotWorker.ts、src/workers/factoryAutopilotWorker.ts	60 秒证据仿真 + 瓶颈/能耗归因移出主线程，串行调度、可取消；关闭开关、切换项目或离开诊断页即清理

边界：巡检不写存档；趋势为固定窗口线性外推而非机器学习；证据副本每轮从初始状态重放，纯运行性损耗不在逐轮差异中体现。回归钉住：autopilot:units（15 用例）。

22.2 生产构建恢复记录

2026-08-25 前完整构建被两个既有 TypeScript 错误阻断：energyAnalysis.ts 未使用的导入、patrolVoice.ts 引用从未创建的 ./apiFlags 模块。已分别通过删除未用导入、改从 ./api 导入并补出 AI_BASE 导出修复；此后 npm.cmd run build（tsc -b + vite build）与全部核心回归恢复全绿。

22.3 DM4310 真机机械臂套件

hardware/dm4310-arm/ 是面向真机的独立硬件交付物，与前端零耦合：

- firmware/dm4310_protocol.h：达妙 MIT 运控模式协议层（16bit 位置 + 12bit 速度/KP/KD/前馈力矩打包解包，FC/FD/FE/FB 特殊命令），纯 C++ 无平台依赖；
- firmware/dm4310_twai.h / dm4310_motor.h：ESP32 TWAI 总线层与带装配方向符号的关节封装；
- firmware/dm4310_arm_sketch/：串口命令台主程序（list/en/dis/zero/clr/p/g/demo），100Hz 控制，任一电机故障自动全臂失能；
- test/protocol_test.cpp：g++ 主机端协议回归（ALL PASS）；
- README 含 BOM、上位机核对项、CAN ID 分配表、接线、七步机械装配、烧录与八步上电调试流程。

关节命名对应场景 panda_joint1..6。边界：固件未经 ESP32 编译烧录与真机验收；电机 P_MAX/V_MAX/T_MAX 因批次而异，必须用达妙上位机核对后修改 range_dm4310()。

22.4 本轮新增回归脚本索引

脚本	npm 别名	覆盖
scripts/simulation-units.ts	sim:units	dir/grid/types/rng/floorConfig 纯函数契约（18 用例）
scripts/autopilot-units.ts	autopilot:units	时序容量、趋势/ETA、劣化注入、报告模板与回退、瓶颈链确定性（15 用例）

23. ForgeLab 社区后端（2026-08-31）

ForgeLab 使用现有 app_user 作为唯一身份源，不创建独立社区账号。前端进入 /forgelab/login 后复用工作台会话令牌，后端每个社区接口都通过 AuthService.currentUser 校验 Authorization: Bearer <token>。

23.1 数据模型与迁移

Flyway V10__create_forgelab_community.sql 建立六张业务表：

表	职责
forgelab_post	帖子正文、板块、作者快照、状态与种子点赞数
forgelab_attachment	图片/存档/资源包元数据和可选 LONGBLOB 文件内容
forgelab_reply	帖子回复正文和作者快照
forgelab_post_like	用户对帖子的唯一点赞关系
forgelab_reply_like	用户对回复的唯一点赞关系
forgelab_notification	收件人、事件类型、关联帖子/回复和已读时间

V11__seed_forgelab_open_posts.sql 写入六条官方示例帖子 和 19 条示例回复；V12__add_forgelab_seed_like_counts.sql 将展示用的历史点赞数作为可追溯种子计数保存，用户新的点赞仍使用关系表记录，取消点赞不会破坏历史展示计数。

23.2 REST 接口

方法	路径	作用
GET	/api/forgelab/posts	读取已发布帖子流、统计、当前用户点赞状态和附件元数据
GET	/api/forgelab/posts/{postId}	读取帖子正文、附件和完整回复列表
POST	/api/forgelab/posts	multipart 发布帖子，可带 image 与 archive 附件
POST	/api/forgelab/posts/{postId}/like	切换当前用户对帖子的点赞
POST	/api/forgelab/posts/{postId}/replies	发布一条回复
POST	/api/forgelab/replies/{replyId}/like	切换当前用户对回复的点赞
GET	/api/forgelab/notifications	读取当前账号通知

方法	路径	作用
POST	/api/forgelab/notifications/{id}/read	标记单条通知已读
POST	/api/forgelab/notifications/read-all	标记当前账号全部通知已读
GET	/api/forgelab/attachments/{id}/download	鉴权下载已上传附件或跳转公共资源

写接口均按当前账号隔离；帖子作者删除账号后保留作者名称快照，避免历史社区内容变成不可读记录。附件名会去除路径部分，上传大小遵守 Spring Boot 80 MB 单文件限制。前端 src/api/forgeLab.ts 将后端响应映射到 Morpicons 社区页面，后端短时不可用时才使用本地展示降级。

23.3 一致性与边界

- 社区后端保存帖子和附件，不逐帧保存工厂仿真状态；工厂存档作为附件上传，工厂正式项目仍由 /api/factories 管理。
- 点赞关系由数据库唯一主键防止同一账号重复点赞；回复、帖子和附件删除关系由外键级联维护。
- 通知只发送给关联作者，不向执行自己操作的用户制造自通知；官方种子帖作者为空账号，因此不会生成虚假的作者通知。
- AI 润色仍是前端可选解释层，不进入社区事实、权限、附件校验或工厂仿真真相源。

ForgeMind 模块用户文档

> 版本：2026-09-02 • 适用 BUILD 0.1.0 / 存档 v6 >> 本文只说明 ForgeMind 主工作台模块的实际使用方式。生态产品的独立入口见 ForgeCloud、ForgeHub、ForgeLab、ForgeMove 和 ForgePass 用户文档。

1. 模块定位

ForgeMind 是 Forge 生态的三维工厂设计、生产配置、确定性仿真和诊断工作台。它适合在方案落地前搭建产线、验证物料流、检查库存和发现结构问题。

ForgeMind 的事实边界是：布局、碰撞、寻路、物料数量和仿真指标由本地确定性规则计算；AI 只能作为可选解释或方案辅助，不能替代这些结论。

2. 进入模块

1. 打开 ForgeMind 官网，点击“进入工作台”或从 ForgePass 登录后选择 ForgeMind。2. 登录后在“开始你的工厂”弹窗中选择“新建空白工厂”或打开已有项目。3. 需要云端保存时保持 Spring Boot 和 MySQL 在线；只做本地编辑、演示或导入时可以使用前端模式。

3. 主工作区

入口	用途
总览	查看工厂状态、KPI、库存和辅助工作区
建造	放置设备、仓储、车辆和传送带
生产	查看产线、设备状态、物料流和产出
诊断	使用 Agent、自动巡检和生成式规划
生产资料	管理机器、物品、配方和仓储资料

总览页的辅助入口包括机械制造、生产路线、物品详情和仓储控制。打开某个工作区时，其他隐藏面板不会继续接收建造快捷键。

4. 推荐使用流程

4.1 先建资料，再摆设备

1. 在“物品详情”建立原料、半成品和成品。2. 在“生产路线”建立输入、输出和加工时长。3. 在“机械制造”建立机器并绑定生产路线。4. 回到“建造”放置设备、仓储和传送带。

4.2 再运行和检查

1. 为入货仓库设置真实库存，为普通货架设置容量和库存。2. 将传送带端口与设备输入/输出方向对齐。3. 运行仿真，观察设备是否加工、线路是否堵塞、物料是否到达出货仓库。4. 在“生产”或“总览”检查消耗、产出、在途批次和设备利用率。

4.3 最后诊断和保存

诊断页先运行只读分析，再根据 Finding 生成 Patch。需要修改存档时必须人工审批；应用后可以继续复核或回滚。确认后结果后使用“手动保存存档”或项目弹窗保存版本。

5. 建造与楼层

- 建造模式支持鼠标放置、旋转、拖绘传送带和 Esc 取消。
- 传送带拖绘会补齐连续格、识别转角，并在合法方向吸附设备端口。
- L1、L2、L3 是独立高度的楼层；当前编辑层可修改，其他开启楼层只作为只读上下文显示。
- 设备不能越过边界，也不能与其他对象占用格重叠。
- 跨层传送使用倾斜传送带或无人机，不要把普通平面传送带当作升降线路。

6. 仓储和车辆

- 入货仓库是供料边界，会记录实际取出和消耗。
- 出货仓库是交付边界，会记录实际入库和产出。
- 普通货架有有限容量；缺货时运输任务等待，不会自动半载或凭空补货。
- AGV 使用同层八方向寻路，可设置起点、终点、物品、每趟数量和库存阈值。
- 无人机支持跨楼层运输，使用三维路径；暂停仿真时运输进度也暂停。

7. 诊断工具怎么选

需要回答的问题	使用入口
哪个端口、碰撞或线路断开	Agent 结构诊断
最近运行窗口是否劣化	自动巡检
目标产能下怎样重排	生成式工厂
某个方案会不会提高产出	What-if 副本仿真

诊断结论需要结合运行证据。没有足够样本时，系统会降低置信度或提示继续观察。

8. 保存与排障

手动保存的是工厂结构、资料和可恢复配置，不是每一帧运行坐标。遇到登录失败先检查 8080；遇到物料不动先检查物品、配方、端口方向和下游容量；遇到导入失败先确认 JSON/GLB 格式与资源归属。

ForgeMind 不直接控制现场设备。需要移动端查看任务、库存或社区内容时使用 ForgeMove；需要共享资源和经验时使用 ForgeLab。

ForgePass 用户文档

> 版本：2026-09-02 • 统一身份入口

1. 模块定位

ForgePass 是 ForgeMind、ForgeCloud、ForgeHub 和 ForgeLab 共用的身份入口。一个账户可以按权限进入不同 Forge 产品，认证过程仍由 ForgeMind Spring Boot 后端完成。

2. 注册

1. 在 ForgeMind、ForgeCloud、ForgeHub 或 ForgeLab 点击进入产品。2. 在 ForgePass 页面选择注册/首次登记。3. 输入账户标识，至少 2 个字符；密码至少 6 位。4. 提交成功后会进入请求产品；需要进入 ForgeMind 工作台时，按提示选择项目。

账户标识不强制要求是邮箱。ForgePass 页面会关闭浏览器的 email-only 校验，但服务端仍会校验账户长度、密码和唯一性。

3. 登录和退出

登录时输入已有账户和密码。成功后页面会保存会话令牌并调用当前用户校验；ForgeCloud 右上角账户菜单提供“退出登录”，退出会清除当前 ForgePass 会话并返回认证入口。

退出后不要继续使用旧页面中的云端操作。重新登录后，系统会根据当前账户重新加载项目、资源、成员和社区权限。

4. 安全提示

- 密码不会以明文保存在数据库中。
- 不要把密码、令牌或 API 地址发布到 ForgeLab 帖子和附件。
- ForgePass 只负责身份，不代表用户拥有所有工作空间和项目权限。
- ForgeCloud、ForgeLab 和 ForgeMove 的数据权限仍由服务端按账户、工作空间和项目校验。

5. 登录失败排查

提示/现象	处理
账户标识至少需要 2 个字符	增加账户标识长度
密码至少需要 6 位	使用符合长度要求的密码
身份验证失败	检查账户、密码和后端 8080
进入产品后没有数据	检查当前工作空间、项目角色和接口状态
退出后仍显示旧内容	刷新页面或重新打开产品入口

ForgeCloud 用户文档

> 版本：2026-09-02 • 适用 ForgeCloud 控制台当前实现

1. 模块定位

ForgeCloud 是 Forge 生态的工业智能云控制台，负责工作空间、项目版本、资源、发布、协作、设备摘要、孪生状态、数据事件和 AI 任务的统一管理。

它保存和组织可追溯的云端事实，不替代 ForgeMind 的三维编辑器和逐 tick 仿真，也不直接控制现场设备。

2. 登录和工作空间

1. 从 ForgeMind 顶部导航、页脚或直接访问 /forgecloud。2. 未登录时进入 ForgePass，使用已有账户登录或注册。3. 登录后从左上角工作空间选择器切换空间；管理员可创建工作空间和管理成员。4. 页面右上角账户菜单可进入成员权限，或执行“退出登录”。

所有云端数据按当前账户和工作空间隔离。没有权限的项目、资源、成员和审计记录不会因为修改浏览器请求而被读取。

3. 首页和主要页面

页面	用户任务
总览	查看当前工厂归档、云端资源、连接、数据库状态和活动队列
项目	创建空白项目、提交版本、查看历史和版本评论
资源	登记 JSON/GLB、查看 manifest、创建不可变新版本和下载
发布	将项目版本写入发布记录，查看审批/任务关联
任务与审批	创建任务、发起审批、批准、拒绝或要求重新规划
设备/孪生/数据	登记设备、查看状态、写入手动遥测和查看孪生绑定
运行与质量	查看运行事件、遥测窗口、工单、质量和维护记录
成员/审计	管理非所有者角色、查看活动并导出当前工作空间 CSV
AI	查看规则任务和确定性执行状态

4. 项目版本

项目版本是不可变记录。进入项目页后创建空白项目，或从 ForgeMind 提交当前存档。提交时系统检查当前版本指纹；如果其他人已经提交新版本，会返回版本冲突，先刷新并重新确认差异。

版本评论、任务和发布记录都绑定到项目版本，不要把“当前工作副本”误认为已经发布的正式版本。

5. 资源和许可证

资源可以登记元数据、JSON 定义、GLB 文件和 manifestVersion=1 的许可证/依赖信息。公开资源必须有 SPDX、许可证表达式或明确声明；缺少许可证的公开资源会被拒绝。

创建新版本不会覆盖旧版本。资源所有者或工作空间管理员可以创建版本，当前版本切换会保留历史、哈希、文件数量和创建人。

下载或复用资源前，先阅读许可证和来源说明。ForgeCloud 的表格展示的是元数据，不返回数据库凭证，也不直接展示业务表行内容。

6. 协作、审批和活动

- 任务和审批都支持客户端幂等键，重复点击不会制造重复业务记录。
- 项目级角色包括 manager、editor、viewer；所有者角色不能被普通成员修改。
- Agent 或移动端产生的请求先进入任务/审批记录，批准后才允许应用受控变更。
- 活动流由审计 Outbox 幂等投影生成；失败事件保留错误摘要并支持重试。

7. 工业数据页面

设备页当前用于设备登记、心跳和命令队列摘要；孪生页用于设备与 ForgeMind 对象或外部资产的绑定；数据页用于数据点和手动遥测事件；运行与质量页用于查看已入库的软件事件。

当前页面不会把演示数据、手动遥测或确定性仿真写成真实 OPC UA/MQTT 现场数据。工业连接仍是只读试点，命令默认进入待处理审计队列，不直接驱动设备。

8. 常见问题

现象	处理
看不到项目	检查当前工作空间和成员角色
提交版本冲突	刷新项目后基于最新版本重新提交
公开资源被拒绝	补充许可证或改为私有资源
数据页没有实时值	先确认是否有已入库遥测事件
页面显示 DEMO DATA	后端不可用，当前是明确标记的本地降级数据

ForgeHub 用户文档

> 版本：2026-09-02 • 当前状态：入口和界面预留

1. 模块定位

ForgeHub 是 Forge 生态中的 3D 资产与模型构建入口，目标是让用户管理模型、查看资源说明，并将可复用资产交给 ForgeCloud 和 ForgeMind 使用。

2. 当前可用范围

1. 从 ForgeMind 官网顶部导航或官方文档页进入 ForgeHub。2. 未登录时通过 ForgePass 统一身份入口登录。3. 登录成功后当前页面展示 ForgeHub 的接口预留状态和产品定位。4. 正式的模型上传、资产发布、版本管理和社区协作尚未在 ForgeHub 页面开放。

当前不要把 ForgeHub 预览页中的展示图当作已经可下载、可发布或可编辑的资产工作区。需要实际导入 GLB 并在工厂里使用时，请回到 ForgeMind 的“资源导入”流程；需要共享资源说明时，可以在 ForgeLab 发布帖子或附件。

3. 未来资产使用建议

准备可复用模型时，建议记录：模型来源、许可证、单位、原点、底面、包围盒、端口方向、适用设备类型和预览图。模型进入 ForgeMind 前仍需经过资源包结构校验和当前用户归属校验。

4. 当前边界

- ForgeHub 当前不执行工厂仿真。
- ForgeHub 当前不修改 ForgeMind 存档。
- ForgeHub 当前不提供现场设备控制。
- ForgeHub 正式资产市场、Fork、依赖治理和发布流程以后续实现为准。

ForgeLab 用户文档

> 版本：2026-09-02 • 适用 ForgeLab 社区当前实现

1. 模块定位

ForgeLab 是 Forge 生态的开放工厂社区，用于阅读和分享工厂存档、模型资源、布局经验、设计经验和设计者公告。它与 ForgeMind、ForgeCloud 共用 ForgePass 账户。

2. 登录和浏览

1. 从官网顶部导航、页脚或 ForgeCloud 入口进入 ForgeLab。2. 未登录时进入 ForgePass；已有工作台会话可以直接通过身份校验。3. 在社区首页使用关键词搜索或板块筛选。4. 点击帖子卡片进入独立文章页，查看正文、作者、统计、相关帖子和附件。

3. 五个板块

板块	适合发布的内容
工厂存档开源	楼层、设备、仓储、配方和可复现运行结果
模型资源开源	GLB、JSON、原点、包围盒、端口和许可证
布局经验分享	网格规划、通道、坡道、避障和维护空间
设计经验分享	目标、约束、取舍和验证过程
设计者公告	版本变化、社区规则和资源治理说明

4. 发布帖子

点击首屏“发布帖子”，依次填写板块、标题、摘要和正文；可以附加图片、工厂存档或资源包。正文建议写清楚 ForgeMind 版本、存档版本、复现条件、已知限制和资源许可证。

AI 润色提示词只用于整理表达，发布前必须人工检查事实、版本、来源和附件内容。帖子与附件在后端可用时写入 ForgeLab 数据库；网络不可用时只保留当前会话草稿并明确提示，不会伪装成已公开内容。

5. 回复、点赞和通知

文章页支持帖子点赞、回复、回复点赞和跳转现场讨论。通知中心会显示回复、点赞和社区事件，可以单条或全部标记已读。回复尽量包含可复现的操作、证据和版本，不要只留下无法检查的结论。

6. 附件和许可

附件下载会经过当前账户权限检查。下载或复用模型、存档和资源包前，先确认作者声明、许可证、来源和是否允许商业使用。社区帖子不是对资产许可的自动授权。

7. 工厂助手

右侧 ForgeLab 工厂助手默认使用确定性规则回答发布、许可、布局和设计问题；启用可选 AI 后只能改善解释，不会替代 ForgeMind 的碰撞、寻路、库存和仿真结论。

ForgeMove 用户文档

> 版本：2026-09-02 • 适用原生微信小程序当前实现

1. 模块定位

ForgeMove 是面向现场工程师、产线负责人和工厂设计者的移动控制台。它用于快速查看工厂状态、任务、库存、监控、档案和 ForgeLab 社区，不替代 ForgeMind 的三维工作台。

2. 打开和登录

1. 使用微信开发者工具导入 ForgeMove/。 2. 在 ForgeMove/config/env.js 配置 Spring Boot 地址；开发阶段可使用 http://127.0.0.1:8080，真机和生产必须使用 HTTPS 合法域名。 3. 点击“进入演示模式”可离线查看完整交互；需要真实数据时使用 ForgePass 账户登录。 4. 在线时小程序会读取认证、工厂、移动摘要、任务和 ForgeLab 接口；接口不可用时显示明确的离线演示标记。

3. 页面说明

页面	用途
工厂总览	Factory Pulse、工厂状态、同步提示和关键指标
产线监控	查看产线运行摘要和异常状态
移动任务	查看现场任务、维护提醒、审批和待办
物料库存	查看库存、低库存预警、在途数量和来源
ForgeLab 社区	浏览动态、求助、经验分享、点赞和草稿
工厂档案	查看项目、楼层、设备、物品和配方摘要
个人设置	查看账户、同步状态、帮助与开源许可

4. 现场任务

任务页用于确认异常、维护提醒和现场待办。在线时创建或更新的任务会经过后端权限和幂等校验；审批动作会进入统一审批记录。离线时可以先保留待办，但同步前要确认网络恢复和当前项目权限。

5. 库存和监控

库存页展示货架、入货仓库、出货仓库和在途摘要。低库存提示用于现场提醒，不会直接修改 ForgeMind 库存。监控页展示移动摘要和最近事件，不等同于三维页面的逐帧坐标或实时仪器读数。

6. 离线边界

- 离线演示数据必须以“离线演示”标记识别。
- 小程序不直接连接 MySQL。

- 小程序不执行 Agent Patch。
- 小程序不修改三维布局、碰撞或寻路结果。
- 需要设计、建造、仿真、诊断、审批和资源导入时，返回 ForgeMind Web 工作台。

7. 常见问题

现象	处理
登录失败	检查 API 地址、Spring Boot 8080 和 HTTPS 合法域名
一直显示离线演示	检查网络、API 地址和当前会话是否过期
看不到工厂	确认账号对项目有访问权限
社区内容不更新	下拉刷新并确认 /api/forgelab/* 可用

Forge 生态模块技术文档

> 版本：2026-09-02 • 当前实现补充

本文档补充 ForgeMind 综合技术文档，集中说明 ForgeMind、ForgePass、ForgeCloud、ForgeHub、ForgeLab 和 ForgeMove 的边界、调用关系和当前实现状态。未落地部分明确标记为预留或路线图，不作为已实现能力。

1. 模块矩阵

模块	当前入口	技术职责	当前状态
ForgeMind	React/Vite /、工作台	三维编辑、资料、确定性仿真、诊断和存档	已落地
ForgePass	ForgePassPage.tsx	注册、登录、会话续登和统一身份入口	已落地
ForgeCloud	React /forgecloud	工作空间、版本、资源、发布、协作、云端事实摘要	已落地；工业生产接入仍受边界约束
ForgeHub	React /forgehub	3D 资产构建和复用入口	接口预留
ForgeLab	React /forgehub、Spring Boot /api/forgehub/*	帖子、附件、回复、点赞和通知	已落地
ForgeMove	原生微信小程序 ForgeMove/	移动摘要、任务、库存、监控和社区入口	已落地；离线和真机网络需部署配置

2. 身份流

ForgeMind / ForgeCloud / ForgeHub / ForgeLab
→ ForgePassPage
→ POST /api/auth/login 或 /api/auth/register
→ Bearer token
→ GET /api/auth/me
→ 产品自身入口和服务端权限检查

ForgePass 不拥有独立用户表；Spring Boot 的 app_user 是身份事实源。密码由服务端使用 BCrypt 处理，会话数据库只保存 token 摘要。退出时前端清除会话并回到认证入口，产品 API 不应继续复用旧 token。

3. ForgeMind 技术边界

ForgeMind 的 src/game/simulation.ts 是固定步长离散生产仿真的唯一真相源；src/scene/ 只负责呈现对象、物料和车辆。src/game/grid.ts 负责边界、足迹、端口、碰撞和吸附规则；src/game/save.ts 负责本地存档 v6 的解析和校验。

Agent、自动巡检和生成式工厂运行在 Worker/确定性规则边界内。AI 不决定布局坐标、碰撞、物料数量、寻路或仿真指标。

4. ForgeCloud 技术边界

ForgeCloud 前端通过 src/api/forgeCloud.ts 访问 /api/v1，服务端按 workspace、项目角色和资源归属执行校验。核心对象包括：

- workspace/member：工作空间和角色；
- project/version：项目工作副本和不可变版本；
- asset/version/blob：资源、manifest、许可证、依赖、哈希和文件本体；
- publication/task/approval：发布、任务和人工决策；
- device/twin/data：设备摘要、孪生绑定和遥测事件；
- activity/outbox：审计事实、幂等投影和失败重试。

版本写入使用乐观并发和客户端幂等键；资源版本使用不可变创建和当前指针切换。当前工业连接为只读/软件回放边界，设备命令默认进入审计队列，不由云端直接执行。

5. ForgeLab 技术边界

Flyway V10–V12 建立帖子、附件、回复、帖子点赞、回复点赞和通知表。ForgeLabController 所有接口都通过 AuthService.currentUser 获取当前账号：

方法	路径	说明
GET	/api/forgelab/posts	帖子流和当前用户点赞状态
GET	/api/forgelab/posts/{id}	正文、附件和回复
POST	/api/forgelab/posts	multipart 发帖
POST	/api/forgelab/posts/{id}/like	切换帖子点赞
POST	/api/forgelab/posts/{id}/replies	发布回复
POST	/api/forgelab/replies/{id}/like	切换回复点赞
GET	/api/forgelab/notifications	当前账号通知
POST	/api/forgelab/notifications/{id}/read	标记已读
GET	/api/forgelab/attachments/{id}/download	鉴权下载

附件保存元数据和可选 LONGBLOB，文件名会去除路径部分并受 Spring Boot 80 MB 单文件限制。社区不保存逐 tick 仿真状态，工厂存档作为附件交给 ForgeMind 重新验证。

6. ForgeMove 技术边界

ForgeMove 使用原生 WXML/WXSS/JavaScript，不直连 MySQL。在线接口包括：

- /api/auth/*：会话和当前用户；
- /api/factories：用户可访问的工厂项目；
- /api/v1/mobile/summary：移动摘要；
- /api/v1/tasks：任务和受权限保护的动作；
- /api/forgelab/*：社区读取、发布和通知。

网络失败时使用明确标记的离线演示数据或待同步状态。移动端不执行 Agent Patch，不改三维坐标，不将摘要指标伪装为三维逐帧事实。

7. ForgeHub 技术边界

当前 /forgehub 通过 ForgePass 后进入产品预览页，保留 forgehub-logo.png 和接口预留状态；尚未建立正式的资产上传、版本、Fork、依赖治理或发布 API。ForgeMind 的 JSON/GLB 资源导入仍是当前可用资产进入工厂的事实入口。

8. 跨模块黄金路径

ForgeMind 建立工厂

- 提交 ForgeCloud 项目版本
- ForgeCloud 记录资源 manifest / 发布 / 审批
- ForgeMove 查看移动摘要和现场任务
- ForgeLab 分享存档、模型和复现说明
- ForgeMind 下载/导入后重新校验、仿真和诊断

每个环节都必须保留来源、版本、许可证和权限边界。ForgeLab 的公共帖子、ForgeMove 的离线摘要和 ForgeCloud 的软件事件不能直接成为 ForgeMind 仿真真相。

9. 验证入口

```
npm.cmd run build
npm.cmd run forgecloud:regression
npm.cmd run forgeweixin:validate
git diff --check
```

ForgeLab 后端变更还需要执行 Maven 测试/打包和真实临时账号回归；ForgeHub 当前只能验证入口、身份保护和预览页，不得宣称资产工作区已完成。